Basi di dati

Corso di Laurea in Ingegneria Informatica ed Elettronica – A.A. 2021-2022

Dipartimento di Ingegneria

docente

Prof. Giuseppe Liotta

Gestione di un centro per l'impiego



studenti

300882	**Piercarmine**	**De Marco**	piercarmine.demarco@studenti.unipg.it
312208	**Redon**	**Karakaci**	redon.karakaci@studenti.unipg.it
311697	**Billal**	**Nadi**	billal.nadi@studenti.unipg.it

Indice

1.Progettazione concettuale **2**
1.1 Raccolta dei requisiti 2
1.2 Analisi delle specifiche 3
1.3 Glossario dei termini principali 5
1.4 Elenco delle operazioni 6
1.5 Schema Entità-Relazione 7
1.6 Vincoli non esprimibili 15
1.7 Dizionario dei dati (Entità) 15
1.8 Dizionario dei dati (Relazioni) 18

2.Progettazione logica **20**
2.1 Tavola dei volumi 20
2.2 Tavola delle operazioni 22
2.3 Analisi delle ridondanze 24
2.4 Eliminazione delle gerarchie 28
2.5 Accorpamento/partizionamento di relationship 30
2.6 Scelta degli identificatori primari 31

3.Modello Relazionale **33**
3.1 Traduzione di Entità 33
3.2 Traduzione di Relazioni 35

4.Implementazione delle operazioni **36**
4.1 Creazione delle tabelle 36
4.2 Creazione delle query 47
4.3 Funzioni utilizzate 55

1.Progettazione concettuale

1.1 Raccolta dei requisiti

Si vogliono rappresentare i dati di interesse di un **Centro per l'impiego** nella città di Perugia. Il Centro per l'impiego è diviso in diversi **uffici**, ognuno dei quali eroga dei servizi specifici. Di tali uffici conosciamo il nome, i dipendenti e il capo ufficio che farà riferimento ad esso.

Il Centro per l'impiego si occupa della fornitura di diversi **servizi**, individuati dal nome e di cui si conosce l'ufficio che li fornisce; ogni ufficio può fornire più servizi. Il servizio fornito dai vari uffici può essere o per il cittadino o per l'azienda.

Il Centro dispone di diverso **personale**, descritto da una matricola, nome, cognome, codice fiscale e stipendio, il quale lavora presso il rispettivo ufficio ad orari stabiliti.

Le aziende e i cittadini si rivolgono al Centro per l'impiego per poter accedere ai vari servizi offerti. Per quanto riguarda le **aziende**, sono identificate dalla partita IVA e sono caratterizzate dal nome, sede legale, dal settore in cui operano, dal numero di dipendenti e dallo stato di attività. Potrebbero essere interessate all'assunzione di personale, mediante **DIA** (Dichiarazione Immediata Assunzione) la quale è identificata da un codice e caratterizzata dalla data di stipula e dal numero massimo di posti disponibili.

Il **cittadino** iscritto al Centro per l'impiego è identificato dal codice fiscale e di esso si conoscono le informazioni personali quali nome, cognome, sesso, data di nascita, indirizzo, recapiti (telefono, email). Oltre ai dati anagrafici si vuole conoscere la sua formazione, in particolare: **titoli di studio**, di cui deve essere riportato il grado il corso e la valutazione; **conoscenze linguistiche**, riportando la lingua e il certificato che attesta il livello più alto in proprio possesso; conoscenze informatiche. I cittadini si possono distinguere in 2 categorie: gli inoccupati, persona che è in cerca della sua prima occupazione e i disoccupati, persona che ha già avuto un'occupazione, ma è momentaneamente senza lavoro. Degli iscritti attualmente disoccupati si vogliono conoscere le esperienze lavorative passate specificando il tipo di lavoro, le conoscenze acquisite, il periodo di lavoro e il motivo della cessazione del rapporto lavorativo. Qualsiasi cittadino, disoccupato o inoccupato, può fare richiesta di impiego presso un'impresa compilando un modello **DID** (Dichiarazione di Immediata Disponibilità) identificato da un codice e caratterizzato dalla data di stipula e dal settore sul quale il cittadino si rende disponibile. Oltre ai normali servizi d'ufficio, i cittadini possono accedere a

dei percorsi formativi di diverso livello, attraverso i quali potranno partecipare a dei **corsi di formazione** dei quali si conosce il nome del corso, la data di inizio e l'ente che lo eroga. Il cittadino potrebbe anche aver già frequentato corsi di formazione, in tal caso, oltre alle informazioni già specificate, si vuole conoscere la data di fine corso e l'esito.

1.2 Analisi delle specifiche

Frasi di carattere generale

Si vuole progettare una base di dati per modellare i dati di interesse di un Centro per l'impiego per gestire l'interazione tra cittadino e azienda nella città di Perugia.

Frasi relative agli uffici

Il Centro per l'impiego è diviso in diversi uffici, ognuno dei quali eroga dei servizi specifici. Di tali uffici conosciamo il nome, il personale e il capo ufficio che farà riferimento ad esso.

Frasi relative ai servizi

Il Centro per l'impiego si occupa della fornitura di diversi servizi, individuati dal nome e di cui si conosce l'ufficio che li fornisce; ogni ufficio può fornire più servizi. Il servizio fornito dai vari uffici può essere o per il cittadino o per l'azienda.

> *Servizio di formazione*
> Il cittadino può richiedere un servizio di formazione con il quale potrà iscriversi a dei corsi specifici forniti da enti esterni.
>
> *Servizio di ufficio*
> Il cittadino può richiedere servizi di ufficio.

Frasi relative al personale

Il Centro dispone di diverso **personale**, identificato da una matricola e di cui conosciamo nome, cognome, codice fiscale e stipendio, il quale lavora presso il rispettivo ufficio ad orari stabiliti.

Frasi relative alle aziende

Le **aziende** si rivolgono al centro per l'impiego per poter accedere ai servizi offerti. Di queste conosciamo la partita IVA, il nome, la sede legale, il settore in cui operano, il numero dei dipendenti e lo stato di attività.

Frasi relative alla DIA

Le aziende che hanno necessità di assumere lavoratori si rivolgono al centro per l'impiego mediante **DIA** (Dichiarazione Immediata Assunzione) la quale è identificata da un codice e caratterizzata dalla data di stipula e dal numero massimo di personale richiesto.

Frasi relative al cittadino

Il **cittadino** iscritto al Centro per l'impiego è identificato dal codice fiscale e di esso si conoscono i dati anagrafici quali nome, cognome, sesso, data di nascita, indirizzo, recapiti (telefono, email) e i dati relativi alla formazione tra cui i titoli di studio, le conoscenze linguistiche e le conoscenze informatiche.

> *Cittadino inoccupato*
> Il cittadino che è in cerca della sua prima occupazione.
>
> *Cittadino disoccupato*
> Il cittadino che ha già avuto un'occupazione, ma è momentaneamente senza lavoro.

Frasi relative ai titoli di studio

Del cittadino si vogliono conoscere i titoli di studio, di cui deve essere riportato il livello, il corso e la valutazione

Frasi relative alle conoscenze linguistiche

Del cittadino si vogliono conoscere le sue conoscenze linguistiche, riportando la lingua e il certificato che attesta il livello più alto in proprio possesso.

Frasi relative alle esperienze lavorative

Delle esperienze lavorative passate specificando il tipo di lavoro, le conoscenze acquisite, il periodo di lavoro e il motivo della cessazione del rapporto lavorativo.

Frasi relative al DID

Qualsiasi cittadino, disoccupato o inoccupato, può fare richiesta di impiego compilando un modello **DID** (Dichiarazione di Immediata Disponibilità) identificato da un codice e caratterizzato dalla data di stipula e dal settore sul quale il cittadino si rende disponibile.

Frasi relative ai corsi

I cittadini possono accedere a dei percorsi formativi di diverso livello, di tali **corsi di formazione** si conosce il nome del corso, la data di inizio e l'ente che lo eroga. Il cittadino potrebbe anche aver già frequentato corsi di formazione, in tal caso, oltre alle informazioni già specificate, si vuole conoscere la data di fine corso e l'esito.

1.3 Glossario dei termini principali

TERMINE	DESCRIZIONE	SINONIMI	TERMINI COLLEGATI
Cittadino	Persona che è iscritto al centro per l'impiego e può richiedere determinati servizi.	Iscritto	Disoccupato, Inoccupato, DID, Servizi
Azienda	Ente interessato all'assunzione di lavoratori e alla richiesta di servizi.	Impresa	Servizi, DIA
Uffici	Uffici specifici del centro per l'impiego che forniscono servizi.		Personale, Servizi DIA, DID
Servizi	Ciò che viene erogato dagli uffici e che permette a cittadini e imprese di effettuare richieste.		Uffici, Corso, Cittadino, Azienda
Personale	Rappresenta gli impiegati del centro per l'impiego.	Dipendenti	Uffici

Corsi	Servizio erogato da enti esterni al centro ai quali il cittadino può partecipare.	Percorso formativo	Cittadino, Servizi

1.4 Elenco delle operazioni

1. Aggiunta di un nuovo cittadino indicando tutti i suoi dati (40 volte al giorno);
2. Aggiunta di una nuova azienda indicando i suoi dati (8 volte alla settimana);
3. Aggiunta di un nuovo ufficio (1 volta all'anno);
4. Aggiunta di un nuovo corso (10 volta ogni due mesi);
5. Aggiunta di un nuovo dipendente indicato i suoi dati (1 volta all'anno);
6. Permettere al cittadino la richiesta di un'occupazione mediante DID (500 volte alla settimana);
7. Permettere alle imprese la ricerca di personale mediante DIA (50 volte alla settimana);
8. Aggiungi esperienza lavorativa al cittadino (30 volte al giorno);
9. Modifica dati relativi al numero dei dipendenti di un'azienda (30 volte alla settimana);
10. Modifica il capo di un ufficio;
11. Modifica dati relativi al personale (1 volta a settimana);
12. Modifica stato di attività di un'azienda (30 volte all'anno);
13. Elimina un cittadino (20 volte alla settimana);
14. Elimina un corso (5 volta ogni due mesi);
15. Elimina un dipendente (1 volta all'anno);
16. Stampa tutti i servizi per i cittadini (30 volte al giorno);
17. Stampa tutti i servizi per le aziende (10 volte al giorno);
18. Per ogni ufficio visualizzare il capo e il numero di impiegati (20 volte alla settimana);
19. Visualizzare la data di fine rapporto lavorativo dei cittadini disoccupati (3 volte alla settimana);
20. Contare il numero di posti lavorativi disponibili in un certo settore (3 volte al giorno);
21. Contare i posti lavorativi disponibili per ogni azienda (3 volte al giorno);
22. Selezionare i titoli di studio dei cittadini in cerca di occupazione per un dato settore (20 volte al giorno);
23. Mostrare codice fiscale, nome e cognome dei cittadini che hanno avuto più esperienze lavorative rispetto agli altri (5 volte al giorno);

24. Mostrare il codice fiscale dei cittadini che hanno richiesto più di 1 servizio nel 2021 (10 volte all'anno);
25. Mostrare l'ufficio che spende di meno in stipendi per il proprio personale ad eccezione del capo (1 volta all'anno);
26. Contare i cittadini in cerca di occupazione per settore (2 volte al giorno);
27. Mostrare i corsi di formazione attualmente disponibili (200 volte al giorno);
28. Visualizzare nome e cognome dei cittadini che hanno frequentato dei corsi (3 volte al giorno);
29. Visualizzare gli orari di lavoro del personale (50 volte al giorno);

1.5 Schema Entità-Relazione

I tre concetti fondamentali per la realizzazione del modello concettuale sono cittadino, azienda e ufficio:

- Il cittadino identifica tutti i cittadini iscritti al centro per l'impiego in cerca di occupazione;
- L'azienda identifica tutte le imprese in cerca di dipendenti.
- L'ufficio gestisce l'interazione tra il cittadino e l'impresa

Inizialmente lo schema scheletro è composto dalle 3 entità principali CITTADINO e AZIENDA che comunicano tramite l'UFFICIO del centro per l'impiego.

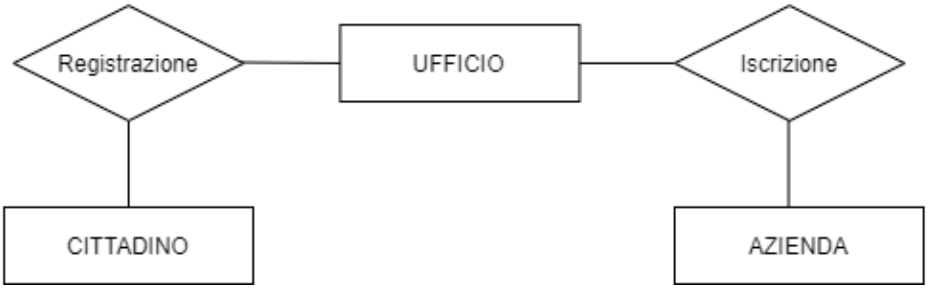


Figura 1 : Schema scheletro (primo livello)

Questo schema naturalmente non permette di rappresentare correttamente tutte le possibili relazioni tra le tre entità, dobbiamo perciò raffinare i concetti.

Dato che qualsiasi cittadino può fare richiesta di impiego presso un'impresa compilando un modello DID (Dichiarazione di Immediata Disponibilità) leghiamo le due entità UFFICIO e CITTADINO mediante un'altra entità chiamata

DID, quindi aggiungiamo la relazione COMPILAZIONEDID che lega CITTADINO a DID e una relazione REGISTRAZIONEDIA che lega DID a UFFICIO. Mentre le aziende che hanno necessità di assumere lavoratori si rivolgono al centro per l'impiego mediante DIA (Dichiarazione Immediata Assunzione), quindi leghiamo l'entità AZIENDA all'entità DIA mediante la relazione REGISTRAZIONEDIA, mentre leghiamo l'entità DIA all'entità AZIENDA tramite la relazione COMPILAZIONEDIA

.

Lo schema che otteniamo a questo punto è il seguente.

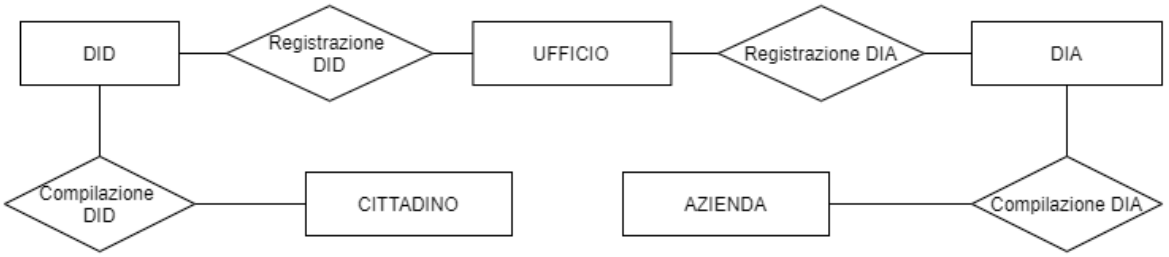


Figura 2 : Evoluzione schema (secondo livello)

I cittadini si possono distinguere in 2 categorie: gli inoccupati, persona che è in cerca della sua prima occupazione e i disoccupati, persona che ha già avuto un'occupazione, ma è momentaneamente senza lavoro. Quindi rappresentiamo queste 2 categorie attraverso una generalizzazione totale ed esclusiva di cittadino che presenta due entità figlie: DISOCCUPATO e INOCCUPATO, poiché non possono esserci cittadini iscritti al centro per l'impiego che non sono né disoccupati, né inoccupati. Degli iscritti attualmente disoccupati si vogliono conoscere le esperienze lavorative passate, quindi dobbiamo rappresentare queste esperienze lavorative mediante un'entità di nome ESPERIENZALAVORATIVA collegata a disoccupato mediante la relazione DICHIARA. Inoltre per classificare i diversi lavori legati alle varie esperienze lavorative, utilizziamo l'entità LAVORO che colleghiamo all'entità ESPERIENZALAVORATIVA tramite la relazione INERENTE.

Lo schema che otteniamo a questo punto è il seguente.

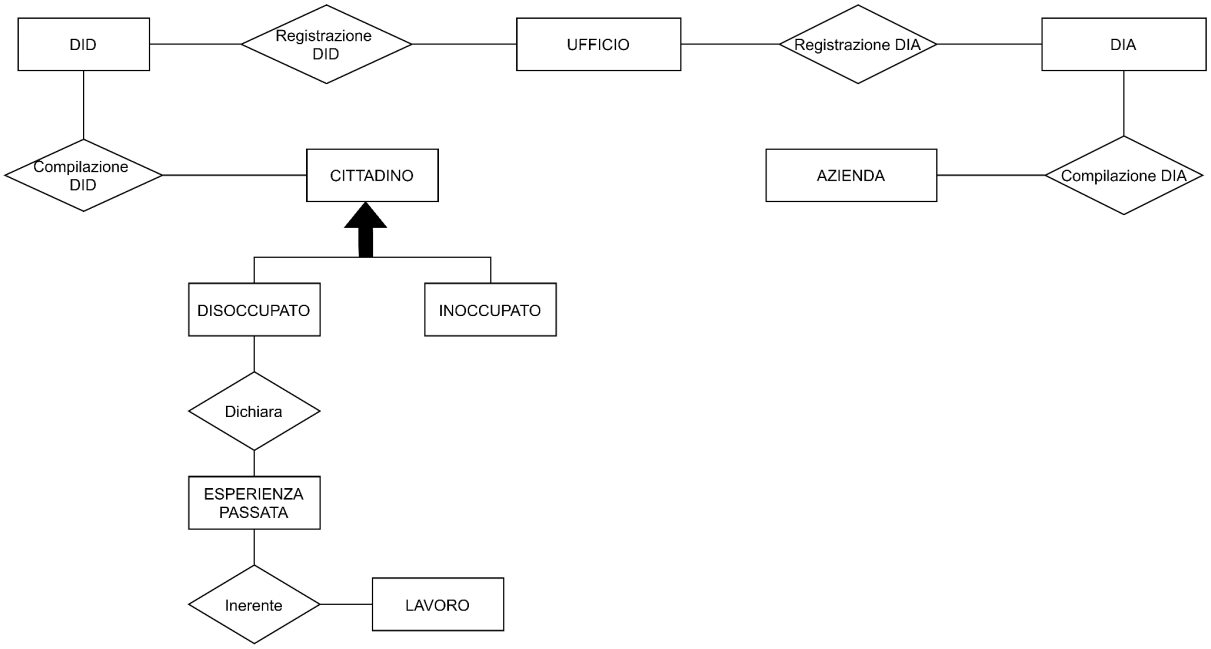


Figura 3 : Evoluzione schema (terzo livello)

Del cittadino iscritto al Centro per l'impiego si vogliono conoscere i dati relativi alla formazione tra cui i titoli di studio che rappresentiamo mediante l'entità TITOLODISTUDIO collegata a cittadino tramite la relazione ATTESTA e le conoscenze linguistiche anch'essa rappresentata da un'entità collegata a cittadino con la relazione CERTIFICA.

Lo schema che otteniamo a questo punto è il seguente.

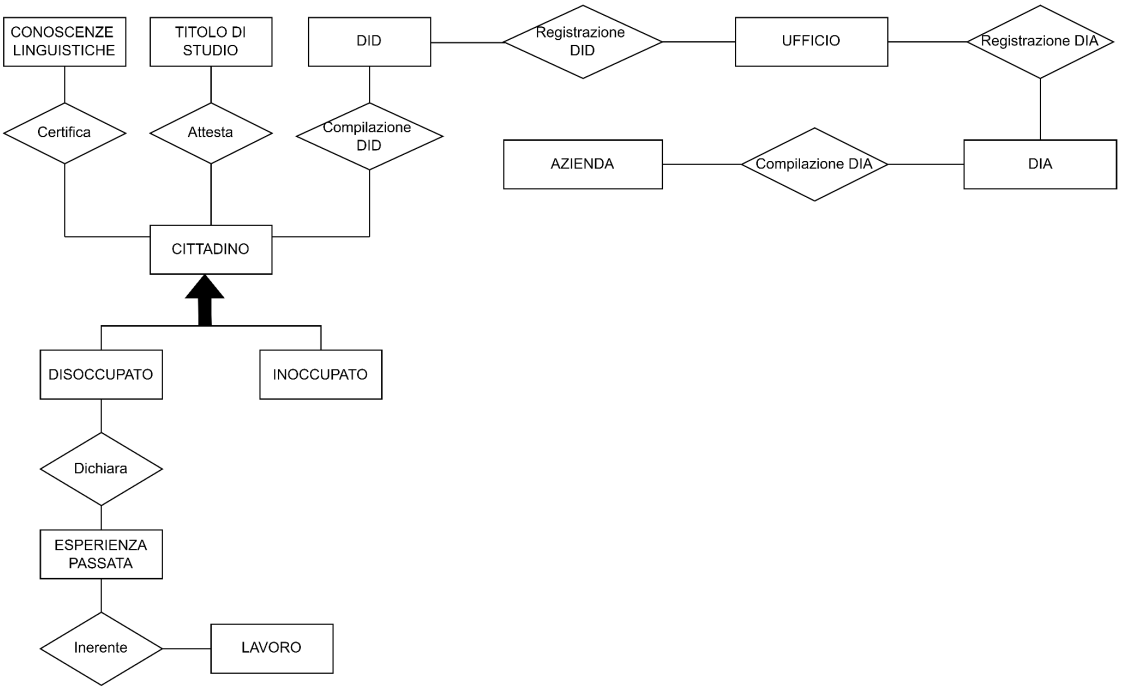


Figura 4 : Evoluzione schema (quarto livello)

Di ogni ufficio conosciamo il personale e il capo che farà riferimento ad esso, perciò abbiamo creato un'entità PERSONALE da collegare ad UFFICIO e dato che ogni ufficio avrà il suo capo, avremo due relazioni che collegano ufficio a personale: la prima è AFFERENZA che indica che ogni membro del personale afferisce al suo ufficio, la seconda è CAPOUFFICIO che indica il fatto che ogni ufficio ha il suo capo.

Lo schema che otteniamo a questo punto è il seguente.

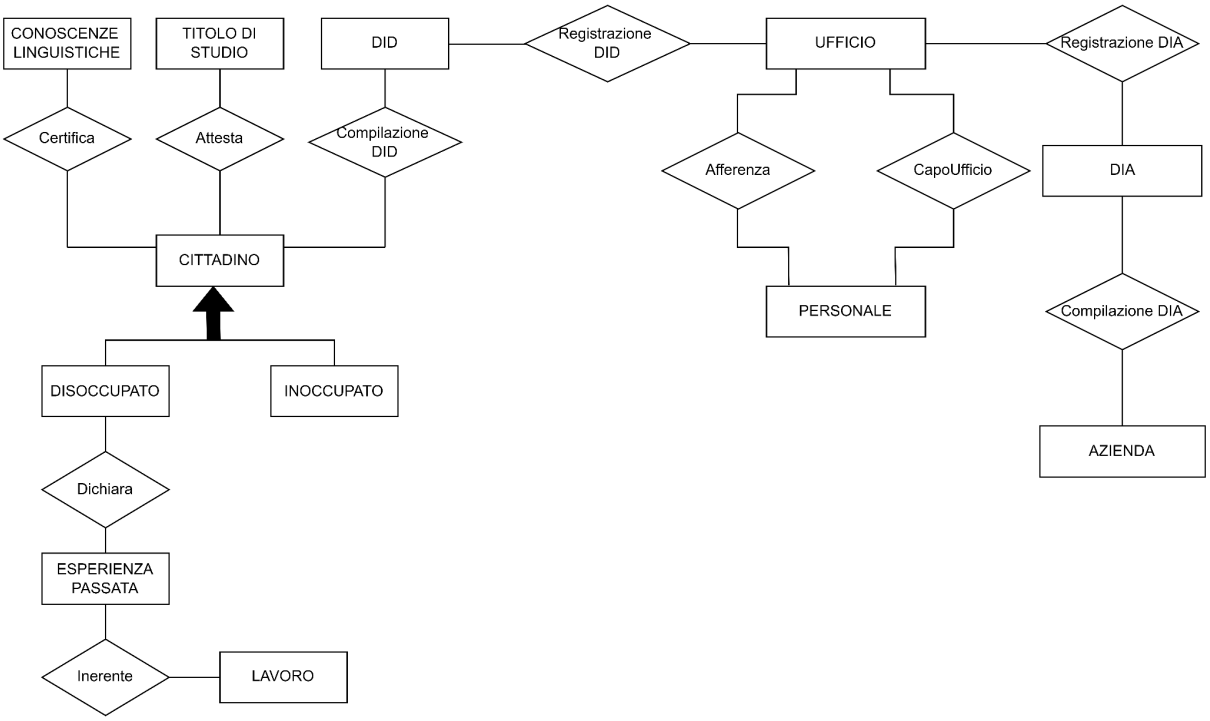


Figura 5 : Evoluzione schema (quinto livello)

Ogni ufficio può fornire più servizi che permettono ai cittadini e alle imprese di effettuare richieste, quindi possiamo creare un'entità SERVIZIO da collegare ad UFFICIO mediante la relazione OFFERTA. Il servizio fornito dai vari uffici può quindi essere o per il cittadino o per l'azienda, di conseguenza possiamo rappresentare queste 2 tipologie di servizi mediante una generalizzazione completa ed esclusiva che presenta due entità figlie: SERVIZIOPERCITTADINO che riporta i soli servizi dedicati al cittadino e SERVIZIOPERAZIENDA che riporta i servizi dedicati all'azienda.
Lo schema che otteniamo a questo punto è il seguente.

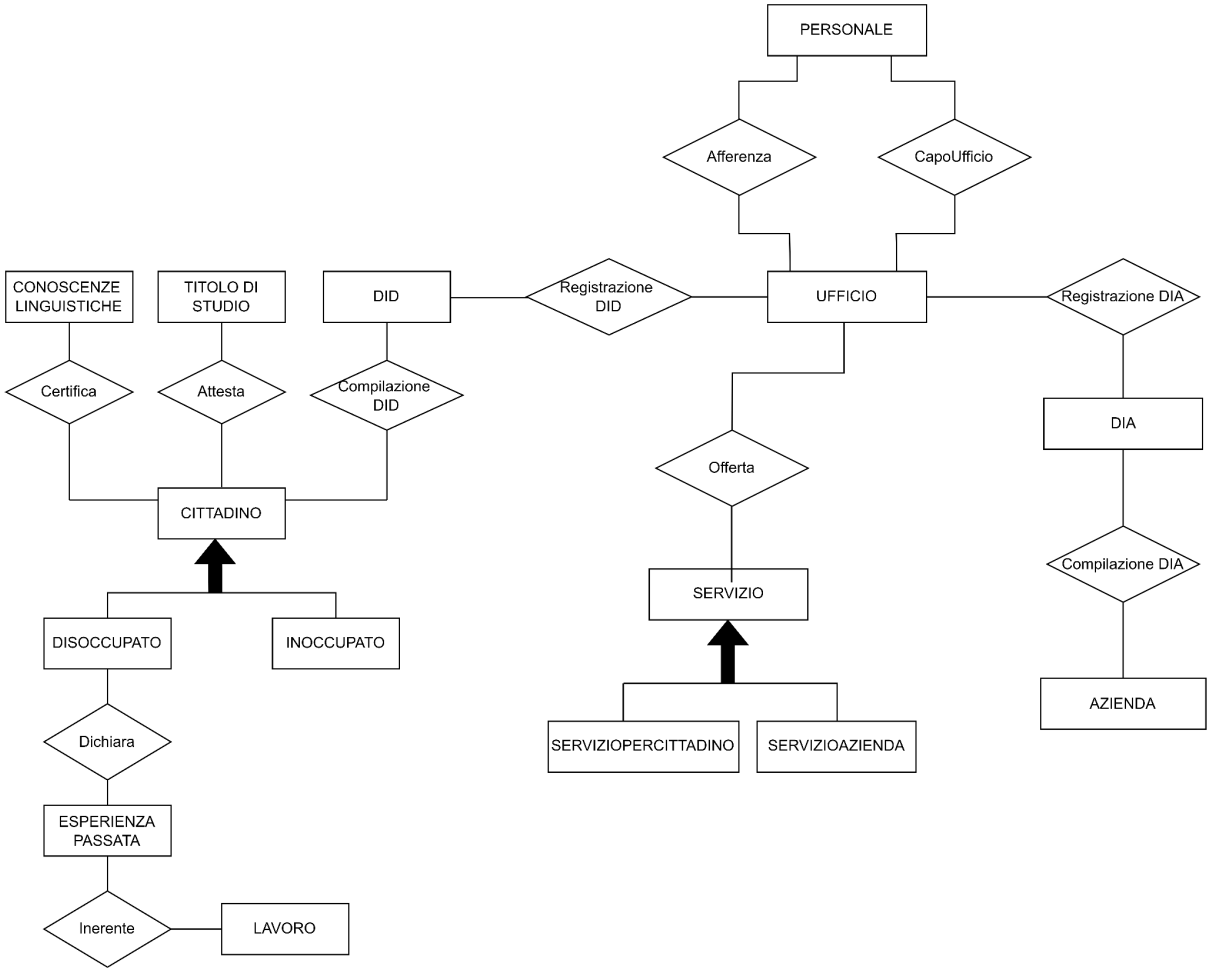


Figura 6 : Evoluzione schema (sesto livello)

Oltre ai normali servizi d'ufficio, i cittadini possono accedere a dei percorsi formativi di diverso livello, attraverso i quali potranno partecipare a dei corsi di formazione. Quindi possiamo effettuare una distinzione tra i servizi d'ufficio e i servizi di formazione dedicati ad esso. Tale distinzione la rappresentiamo mediante una generalizzazione completa ed esclusiva che presenta le due entità figlie: SERVIZIODIFORMAZIONE e SERVIZIODIUFFUCIO. Se collegassimo l'entità SERVIZIODIUFFICIO a CITTADINO mediante una relazione RICHIESTASERVIZIOCITTADINO, un determinato cittadino potrebbe effettuare la richiesta di un determinato servizio solo una volta.. Per ovviare questo problema si è deciso di trasformare l'associazione RICHIESTASERVIZIOCITTADINO con un'omonima entità, introducendo le necessarie associazioni: ESEGUE che la collega a cittadino e l'associazione ATTINENTE che lo collega a SERVIZIODIUFFICIO. Queste due associazioni sono dunque servite per definire un modello concettuale in grado di tener conto di molteplici richieste di un certo servizio da parte dello stesso cittadino. Per quanto riguarda i corsi di formazione, abbiamo creato un'entità CORSO che si lega a SERVIZIODIFORMAZIONE mediante la relazione erogazione e ad ufficio tramite 2 relazioni:

PARTECIPAZIONEPASSATA poichè Il cittadino potrebbe anche aver già frequentato dei corsi di formazione, quindi dobbiamo poter tener traccia di tali partecipazioni e PARTECIPAZIONEPRESENTE per tener traccia delle attuali partecipazioni ai corsi di formazione.

Lo schema che otteniamo a questo punto è il seguente.

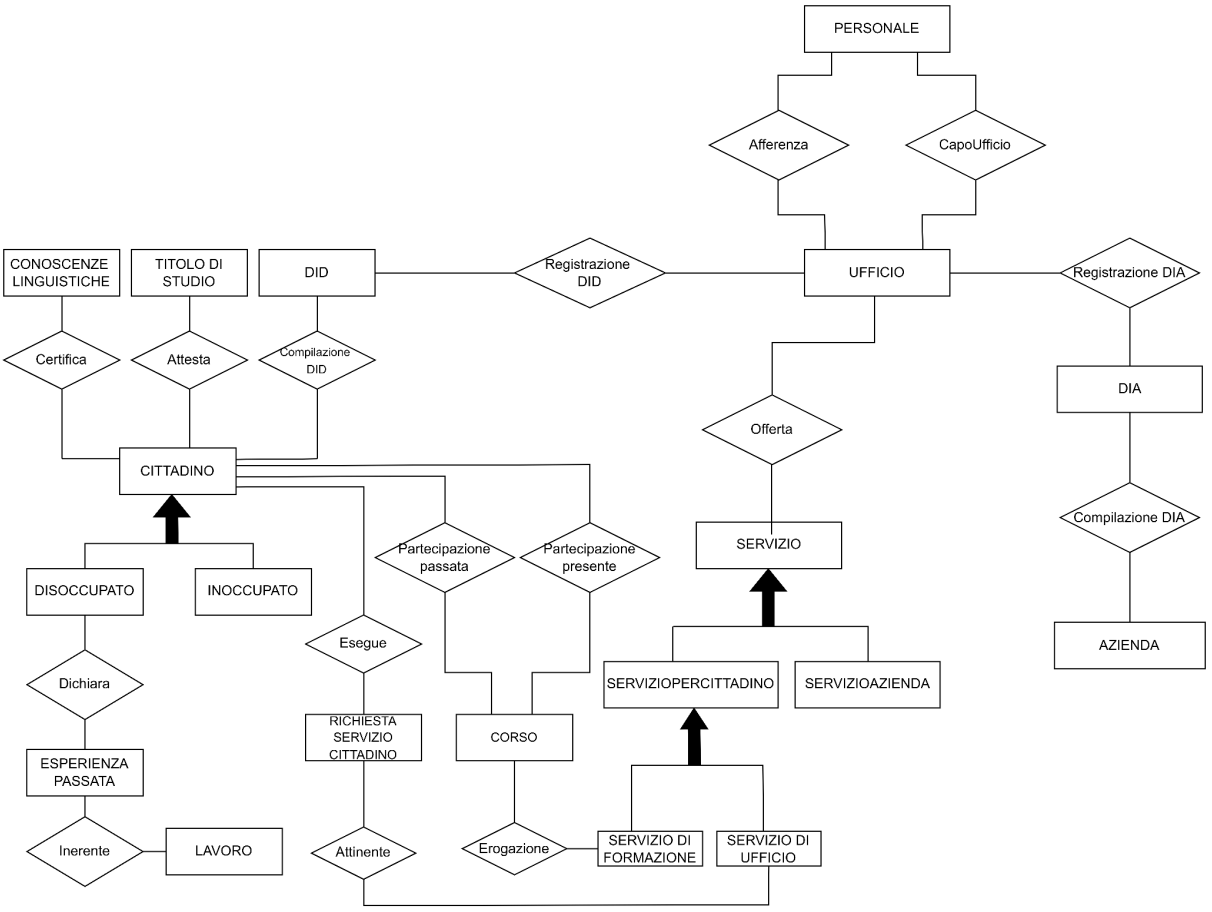


Figura 7 : Evoluzione schema (settimo livello)

Per quanto riguarda i servizi per l'azienda Inizialmente l'entità RICHIESTASERVIZIOAZIENDA era un'associazione che coinvolgeva le entità AZIENDA e l'entità SERVIZIOAZIENDA: in questo modo una determinata azienda poteva effettuare la richiesta di un determinato servizio una sola volta. Per ovviare questo problema si è deciso di trasformare l'associazione RICHIESTASERVIZIOAZIENDA con un'omonima entità, introducendo l'associazione EFFETTUA che la collega ad AZIENDA e l'associazione RELATIVO che la collega a SERVIZIOAZIENDA . Queste due associazioni sono dunque servite per definire un modello concettuale in grado di tener conto di molteplici richieste di un certo servizio da parte della stessa azienda.

Di seguito lo schema con queste ultime aggiunte.

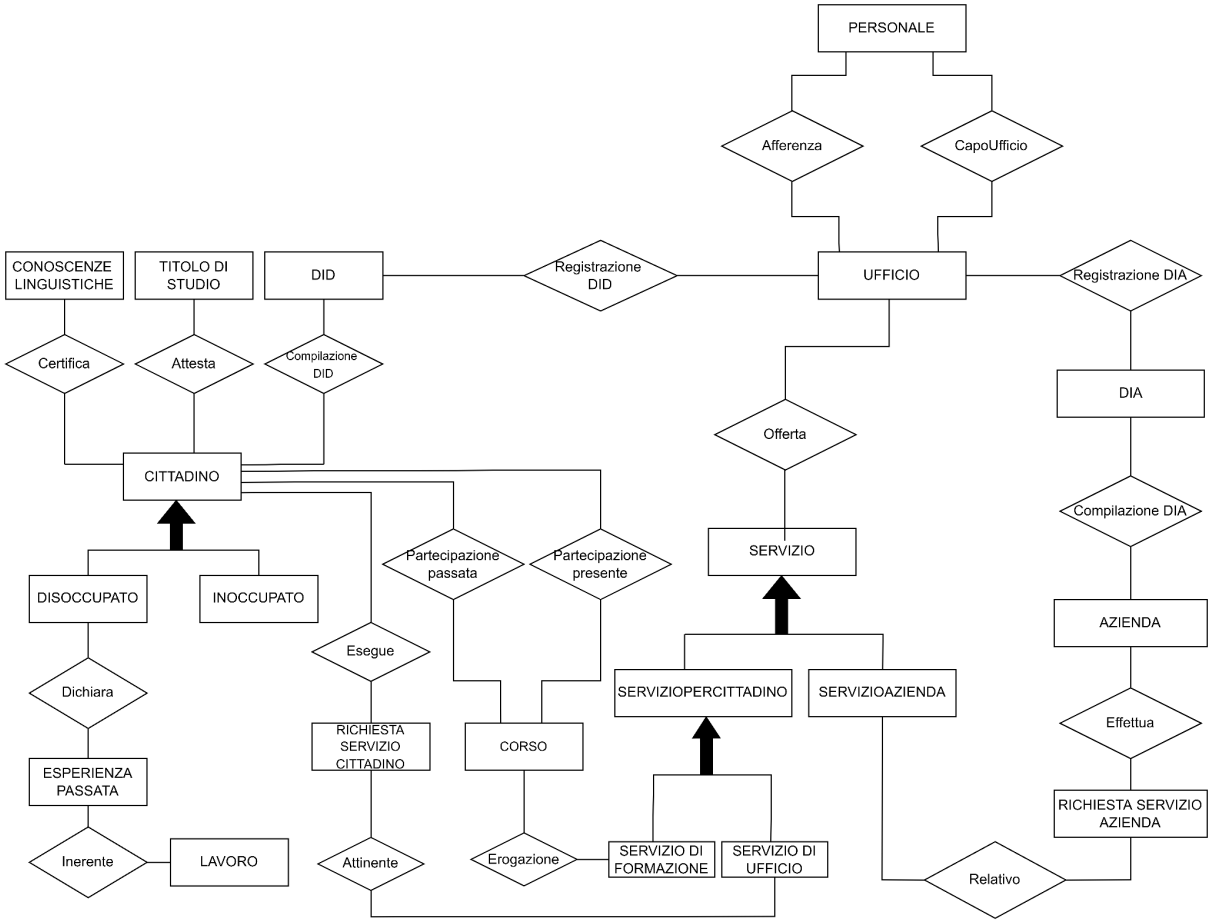


Figura 8 : Evoluzione schema (ottavo livello)

Per terminare la fase di progettazione concettuale è necessario completare lo schema di figura 8 con alcune informazioni qualitative e quantitative, ovvero con gli attributi e le cardinalità delle relazioni. La maggior parte di queste informazioni sono ricavate ancora una volta dall'analisi delle specifiche.

Lo schema E – R completo viene mostrato in figura 9.

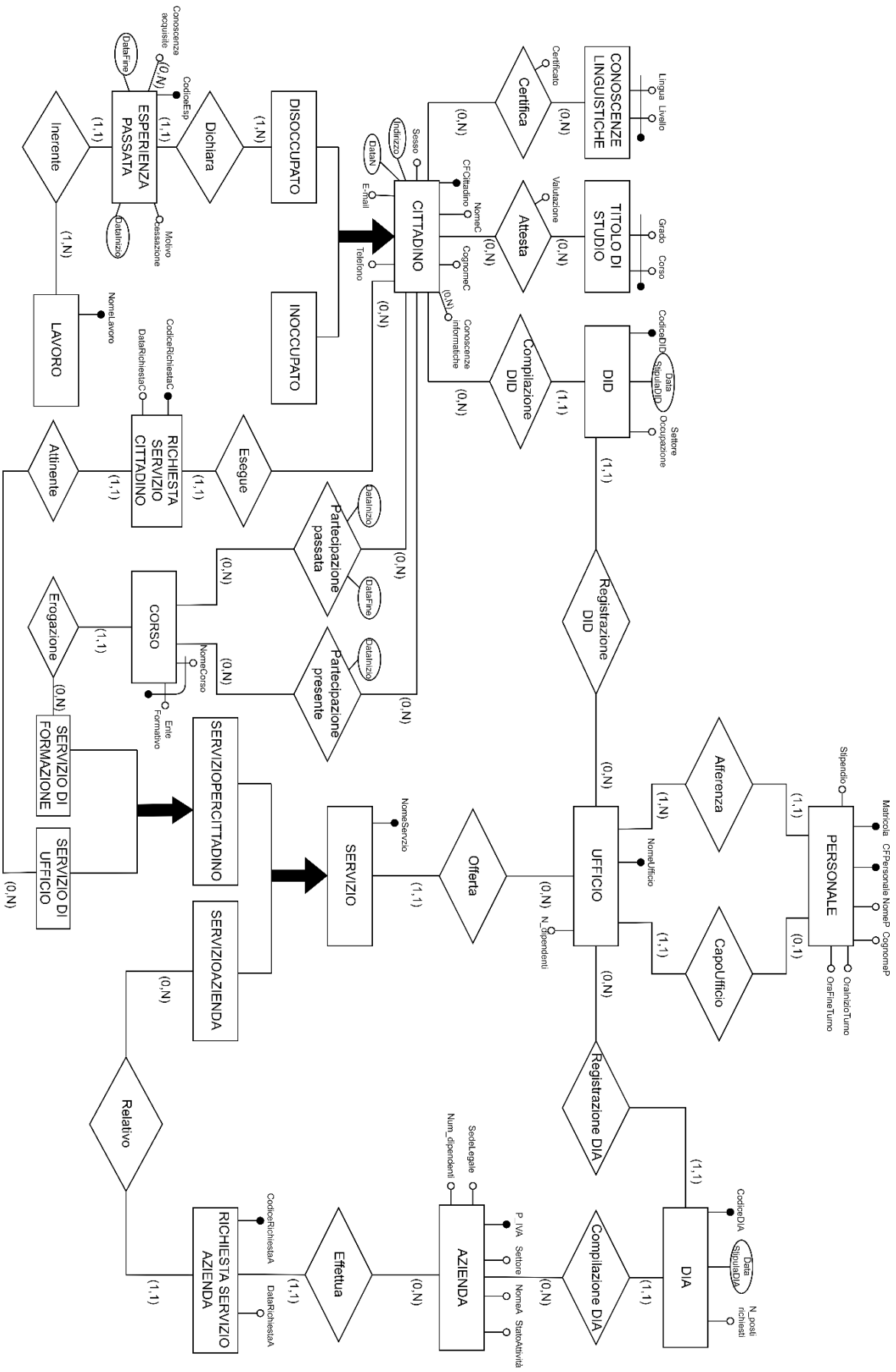


Figura 9 : Schema completo

1.6 Vincoli non esprimibili

1. All'iscrizione al centro per l'impiego il cittadino deve avere un'età compresa tra i 18 e i 65 anni;
2. Il capo di un ufficio afferisce ad esso;
3. L'attributo DataFineCorso dell'entità Corso deve essere successivo all'attributo DataInizioCorso della stessa;
4. La data InizioDisoccupazione deve essere precedente o uguale alla data corrente;
5. La data StipulaDIA deve essere precedente o uguale alla data corrente;
6. La data StipulaDID deve essere precedente o uguale alla data corrente;
7. La data DataRichiestaA deve essere precedente o uguale alla data corrente;
8. La data DataRichiestaC deve essere precedente o uguale alla data corrente;
9. OrarioInizioTurno deve essere precedente a OrarioFineTurno;
10. Le occorrenze di DIA possono far riferimento ai soli uffici il cui nome inizia per 'Assunzione';
11. Le occorrenze di DID possono far riferimento ai soli uffici il cui nome inizia per 'Assunzione'.

1.7 Dizionario dei dati (Entità)

Entità	Identificatore	Attributi	Descrizione
PERSONALE	Matricola, CFPersonale	Stipendio, NomeP, CognomeP, OraInizioTurno, OraFineTurno	Rappresenta gli impiegati del centro per l'impiego.
UFFICIO	NomeUfficio	N_abitanti	Uffici specifici del centro per l'impiego.
DID	CodiceDID	DataStipulaDID, SettoreOccupazione	Ciò che consente al cittadino di rendersi disponibile ad incarichi lavorativi.
TITOLO DI STUDIO	Grado, Corso		Rappresenta i titoli di studio del cittadino
CONOSCENZE LINGUISTICHE	CodLingua	Livello, Lingua	Rappresenta le conoscenze linguistiche del cittadino

CITTADINO	CFCittadino	NomeC, CognomeC, Sesso, Indirizzo, E-mail, Telefono, ConoscenzeInformatiche, DataN	Colui che è iscritto al centro per l'impiego e può richiedere determinati servizi a lui dedicati.
DISOCCUPATO	CFCittadino		Sottocategoria di cittadino: è colui che risulta non avere un'occupazione, ma che ha già esperienze lavorative passate
INOCCUPATO	CFCittadino		Sottocategoria di cittadino: è colui che risulta non avere un'occupazione e che non ha mai avuto un'esperienza lavorativa
ESPERIENZA LAVORATIVA	CodEsp	ConoscenzeAcquisite, MotivoCessazione, Ruolo, DataInizio, DataFine	Rappresenta le esperienze lavorative passate della cittadino disoccupato
SERVIZIO DI UFFICIO	Nomeservizio		Sottocategoria dei servizi per il cittadino: individua i normali servizi di ufficio che non permettono l'erogazione dei corsi.
CORSO	NomeCorso, EnteFormativo		Servizio erogato da enti esterni al centro ai quali il cittadino può partecipare.
SERVIZIO DI FORMAZIONE	NomeServizio		Sottocategoria dei servizi per il cittadino: individua

			i servizi che permettono l'erogazione dei corsi.
RICHIESTA SERVIZIO CITTADINO	CodRC	DataRichiestaC	Ciò che permette al cittadino di richiedere servizi.
DIA	CodiceDIA	DataStipulaDIA, N_PostiRichiesti	Ciò che consente alle imprese di rendersi disponibili ad assumere lavoratori.
AZIENDA	P_Iva	Settore, NomeA, SedeLegale, Num_dipendenti, StatoAttività	Ente interessato all'assunzione di lavoratori e alla richiesta di servizi ad esso dedicati
RICHIESTA SERVIZIO AZIENDA	CodRA	DataRichiestaA	Ciò che permette all'azienda di richiedere servizi.
SERVIZIO	NomeServizio		Ciò che viene erogato dagli uffici e che permette a cittadini e imprese di effettuare richieste.
SERVIZIO PER CITTADINO	NomeServizio		Sottocategoria di servizio: individua i servizi rivolti al cittadino.
SERVIZIO AZIENDA	NomeServizio		Sottocategoria di servizio: individua i servizi rivolti alle aziende
LAVORO	NomeLavoro		Rappresenta tutti i possibili lavori che un cittadino può aver fatto

1.8 Dizionario dei dati (Relazioni)

Entità	Entità	Attributi	Descrizione
AFFERENZA	Personale, Ufficio		Indica che ogni membro del personale afferisce ad un ufficio.
CAPO UFFICIO	Personale, Ufficio		Indica che ogni ufficio ha un capo.
EFFETTUA	Azienda, Richiesta Servizio Azienda		Associa ogni azienda alla richiesta di servizio che effettua.
RELATIVO	Servizio Azienda, Richiesta Servizio Azienda		Associa la richiesta di un servizio di un'azienda al servizio corrispondente.
OFFERTA	Ufficio, Servizio		Indica che un ufficio può offrire un certo numero di servizi.
REGISTRAZIONE DIA	Ufficio, DIA		Indica la registrazione di un modulo DIA presso un determinato ufficio.
COMPILAZIONE DIA	Azienda, DIA		Indica la compilazione di un modulo DIA da parte di un'azienda.
REGISTRAZIONE DID	Ufficio, DID		Indica la registrazione di un modulo DID presso un determinato ufficio.
COMPILAZIONE DID	Cittadino, DID		Indica la compilazione di un modulo DID da parte di un cittadino.

PARTECIPAZIONE PASSATA	Cittadino, Corso	DataInizio, DataFine	Indica che ogni cittadino può aver partecipato ad un certo numero di corsi.
PARTECIPAZIONE PRESENTE	Cittadino, Corso	DataInizio	Indica che ogni cittadino può partecipare ad un certo numero di corsi.
EROGAZIONE	Servizio di Formazione, Corso		Associa un corso al servizio che lo eroga.
ATTINENTE	Servizio di Ufficio, Richiesta Servizio Cittadino		Associa la richiesta di un servizio di un cittadino al servizio corrispondente.
ESEGUE	Richiesta Servizio Cittadino, Cittadino		Associa ogni cittadino alla richiesta di servizio che effettua.
DICHIARA	Disoccupato, Esperienza Passata		Indica che i cittadini disoccupati possono avere più esperienze lavorative passate.
INERENTE	Esperienza Passata, Lavoro		Rappresenta il lavoro inerente all'esperienza passata.
CERTIFICA	Conoscenze Linguistiche, Cittadino	Certificato	Indica le conoscenze linguistiche di ogni cittadino con relativo certificato.
ATTESTA	Titolo di Studio, Cittadino	Valutazione	Indica i titoli di studio di ogni cittadino.

2.Progettazione logica

2.1 Tavola dei volumi

Concetto	Tipo	Volume
CITTADINO	E	35000
AZIENDA	E	3000
SERVIZIO	E	50
TITOLO DI STUDIO	E	200
DID	E	in media 3(did) x 35000(cittadini)=105000
DISOCCUPATO	E	25000
INOCCUPATO	E	10000
ESPERIENZA PASSATA	E	in media 3(lavori) x 25000 (cittadini disoccupati)= 75000
CONOSCENZE LINGUISTICHE	E	7
RICHIESTA SERVIZIO CITTADINO	E	in media 4(richieste) x 35000 (cittadini) = 140000
SERVIZIO PER CITTADINO	E	30
SERVIZIO DI FORMAZIONE	E	1
SERVIZIO DI UFFICIO	E	29
CORSO	E	200
UFFICIO	E	30
PERSONALE	E	in media 5(persone) x 30 (uffici)=150
DIA	E	in media 20(did) x 3000(cittadini)=60000

RICHIESTA SERVIZIO AZIENDA	E	in media 3(richieste) x 3000 (aziende) = 9000
SERVIZIO AZIENDA	E	20
LAVORO	E	5000
CERTIFICA	R	35000(cittadini) x 7 (conoscenze linguistiche)=245000
ATTESTA	R	35000(cittadini) x 200 (titoli di studio) = 7000000
COMPILAZIONE DID	R	105000
DICHIARA	R	75000
INERENTE	R	75000
ESEGUE	R	140000
PARTECIPAZIONE PASSATA	R	35000 (cittadini) x 200 (corsi)= 7000000
PARTECIPAZIONE PRESENTE	R	35000
ATTINENTE	R	140000
EROGAZIONE	R	200
REGISTRAZIONE DID	R	105000
AFFERENZA	R	150
CAPOUFFICIO	R	30
OFFERTA	R	50
REGISTRAZIONE DIA	R	60000
COMPILAZIONE DIA	R	60000
EFFETTUA	R	9000
RELATIVO	R	9000

2.2 Tavola delle operazioni

Operazione	Tipo	Frequenza
OP1: Aggiunta di un nuovo cittadino indicando tutti i suoi dati	I	40 al giorno
OP2: Aggiunta di una nuova azienda indicando i suoi dati	I	8 alla settimana
OP3: Aggiunta di un nuovo ufficio	B	1 volta all'anno
OP4: Aggiunta di un nuovo corso	B	10 ogni 2 mesi
OP5: Aggiunta di un nuovo dipendente indicando i suoi dati	B	1 volta all'anno
OP6: Permettere al cittadino la richiesta di un'occupazione mediante DID	I	500 alla settimana
OP7: Permettere alle imprese la ricerca di personale mediante DIA	I	50 alla settimana
OP8: Aggiungi esperienza lavorativa al cittadino	I	30 al giorno
OP9: Modifica dati relativi al numero dei dipendenti di un'azienda	I	30 alla settimana
OP10: Modifica il capo di un ufficio	I	1 volta all'anno
OP11: Modifica dati relativi al personale	B	1 alla settimana

OP12: Modifica stato di attività di un'azienda	B	30 all'anno
OP13: Elimina un cittadino	I	20 alla settimana
OP14: Elimina un corso	B	5 ogni 2 mesi
OP15: Elimina un dipendente	B	1 all'anno
OP16: Stampa tutti i servizi per i cittadini	I	30 al giorno
OP17: Stampa tutti i servizi per le aziende	I	10 al giorno
OP18: Per ogni ufficio visualizzare il capo e il numero di impiegati	I	20 al giorno
OP19: Visualizzare la data di fine rapporto lavorativo dei cittadini disoccupati	B	3 alla settimana
OP20: Contare il numero di posti lavorativi disponibili in un certo settore	I	3 al giorno
OP21: Contare i posti lavorativi disponibili per ogni azienda	I	3 al giorno
OP22: Selezionare i titoli di studio dei cittadini in cerca di occupazione per un dato settore	I	20 alla settimana
OP23: Mostrare codice fiscale, nome e cognome dei cittadini che hanno avuto più esperienze lavorative rispetto agli altri	I	5 al giorno

OP24: Mostrare il codice fiscale dei cittadini che hanno richiesto più di un servizio nel 2021	B	10 all'anno
OP25: Mostrare l'ufficio che spende di meno in stipendi per il proprio personale ad eccezione del capo	B	1 all'anno
OP26: Contare i cittadini in cerca di occupazione per settore	I	2 al giorno
OP27: Mostrare i corsi di formazione attualmente disponibili	I	200 al giorno
OP28: Visualizzare nome e cognome dei cittadini che hanno frequentato dei corsi	I	3 al giorno
OP29: Visualizzare gli orari di lavoro del personale	I	50 al giorno

2.3 Analisi delle ridondanze

Una possibile ridondanza è quella riguardante il numero di dipendenti di un determinato ufficio. Questo valore, infatti, potrebbe essere ricavato indirettamente conteggiando le occorrenze delle relazione Afferenza.

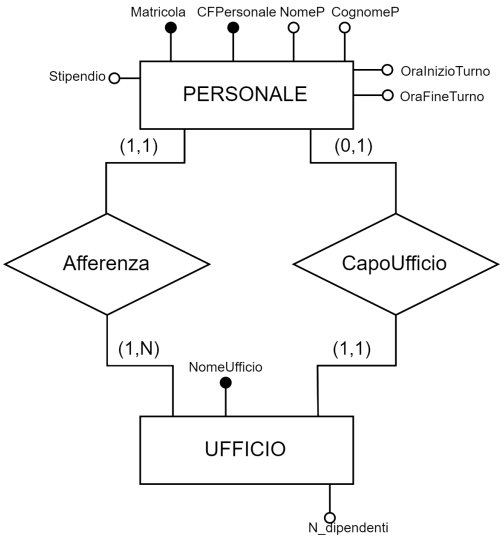


Figura 10 : Schema con ridondanza

Per determinare se conviene o meno mantenere questa ridondanza occorre analizzare e quindi confrontare gli indici di prestazione nei due casi, con e senza ridondanza. L'indice di prestazione viene calcolato sulla base delle operazioni che coinvolgono relazioni ed entità, oggetto della ridondanza.

In questo caso specifico, le operazioni di cui vanno valutati gli accessi sono:

OP4: Aggiunta di un nuovo dipendente indicando i suoi dati (1 volta all'anno);
OP13: Elimina un dipendente (1 volta all'anno);
OP16: Per ogni ufficio visualizzare il capo e il numero di impiegati (20 volte alla settimana);

Cominciamo con l'analizzare gli accessi in presenza di ridondanza, cioè quando è presente in Ufficio l'attributo N_dipendenti.

Tavola degli accessi per **OP4**:

Concetto	Costrutto	Accessi
Personale	Entità	1 Scrittura
Afferenza	Relazione	1 Scrittura
Ufficio	Entità	1 Lettura
Ufficio	Entità	1 Scrittura

Tavola degli accessi per **OP13**:

Concetto	Costrutto	Accessi
Personale	Entità	1 Scrittura
Afferenza	Relazione	1 Scrittura
Ufficio	Entità	1 Lettura
Ufficio	Entità	1 Scrittura

Tavola degli accessi per **OP16**:

Concetto	Costrutto	Accessi
Ufficio	Entità	30 Lettura
CapoUfficio	Relazione	30 Lettura
Personale	Entità	30 Lettura

Considerando il peso di una scrittura pari a 2 letture e moltiplicando i risultati parziali per la frequenza delle operazioni, otteniamo che il costo totale all'anno (OP4 = 1 volta all'anno, OP13 = 1 volta all'anno, OP16 = 960 volte al'anno)
in termini di accessi, in presenza di ridondanza, è dato da:

(3S+1L) + (3S+1L) + (90L x 960) = 7L + 7L + 90L x 960= 86414 letture

Risulta molto evidente che il costo delle operazioni 4 e 13 è trascurabile, si può pertanto dire che il numero di accessi è pari a 86400 letture.

Valutiamo ora i costi in assenza di ridondanza. Lo schema rimane lo stesso, ad eccezione dell'attributo N_dipendenti, che non è più presente.

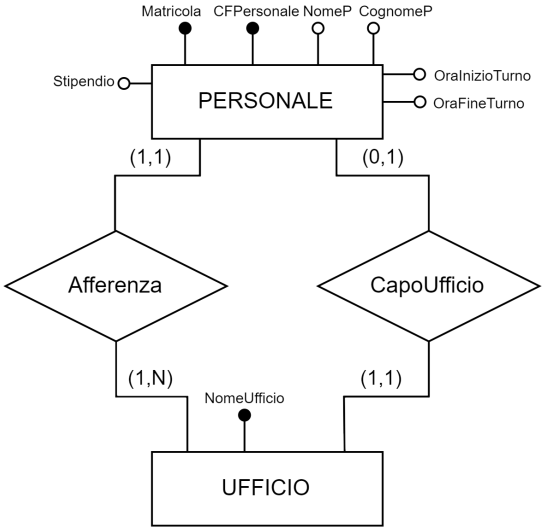


Figura 11 : Schema senza ridondanza

Tavola degli accessi per **OP4**:

Concetto	Costrutto	Accessi
Personale	Entità	1 Scrittura
Afferenza	Relazione	1 Scrittura

Tavola degli accessi per **OP13**:

Concetto	Costrutto	Accessi
Personale	Entità	1 Scrittura
Afferenza	Relazione	1 Scrittura

Tavola degli accessi per **OP16**:

Concetto	Costrutto	Accessi
Ufficio	Entità	30 Lettura
CapoUfficio	Relazione	30 Lettura
Personale	Entità	30 Lettura
Afferenza	Relazione	150 letture

Per l'operazione 16, si considerano in media 5 accessi in lettura per la relazione Afferenza (ottenuto dividendo il numero di personale per il numero di uffici) per calcolare il numero di dipendenti di un ufficio. Questo poi va moltiplicato per il numero di uffici.
Il costo totale in termini di accesso eliminando la ridondanza è dato da:

$$2S + 2S + (90L+30x5L) \times 960 = 4L + 4L + 240L \times 960 = 230408 \text{ letture}$$

Risulta molto evidente che il costo delle operazioni 4 e 13 è trascurabile, si può pertanto dire che il numero di accessi è pari a 230400 letture.

Possiamo quindi concludere che si guadagna in efficienza mantenendo la ridondanza.

2.4 Eliminazione delle gerarchie

Lo schema finale ottenuto presenta due macro generalizzazioni su più livelli: una riguardante l'entità CITTADINO e una l'entità SERVIZIO. Per sostituirle partiamo inizialmente con l'analizzare quelle più interne, fino ad arrivare all'entità in cima alla gerarchia.
Iniziamo l'analisi della generalizzazione dell'entità SERVIZIOPERCITTADINO procedendo con l'accorpamento dell'entità genitore nelle entità figlie, poiché le operazioni si riferiscono solo a occorrenze delle entità figlie è conveniente accorpare l'entità padre nelle entità figlie. La figura seguente mostra lo schema prima e dopo l'analisi.

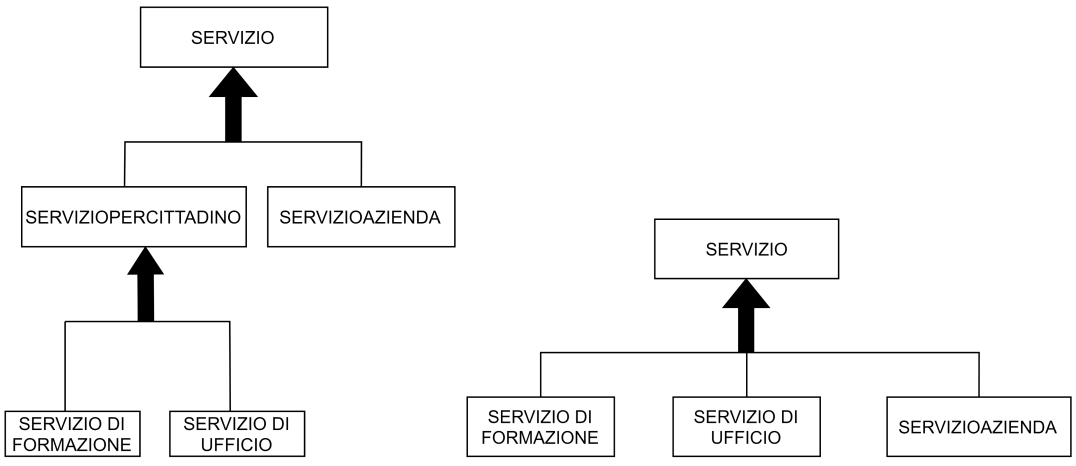


Figura 12: Analisi generalizzazione di SERVIZIOPERCITTADINO

Si procede con il valutare il livello superiore della generalizzazione, dove in questo caso non è possibile eliminare né il padre né le figlie, poiché hanno entrambi interesse ai fini della base di dati, per cui introduciamo delle relazioni tra padre e figlie.

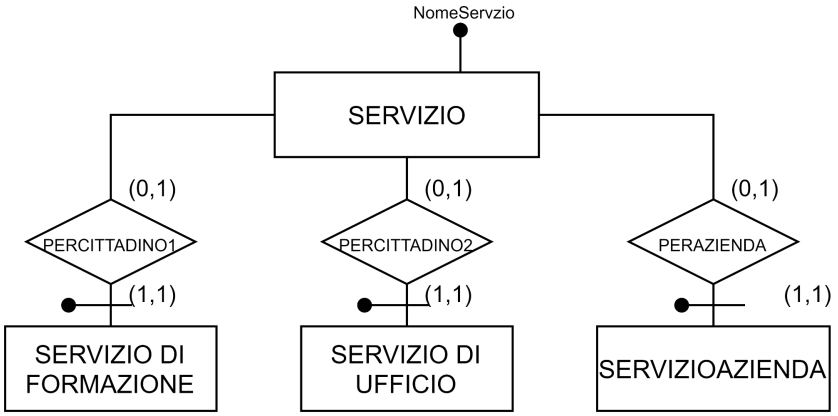


Figura 13 : Schema ristrutturato per la generalizzazione su SERVIZIO

Ora analizziamo la generalizzazione che riguarda l'entità CITTADINO. In questo caso si opta per una soluzione ibrida, ovvero abbiamo scelto di mantenere l'entità DISOCCUPATO in quanto abbiamo delle operazioni che la coinvolgono e che la differiscono da INOCCUPATO. Quindi aggiungiamo un attributo di nome "stato" all'interno dell'entità CITTADINO che indica se il cittadino è disoccupato o inoccupato.

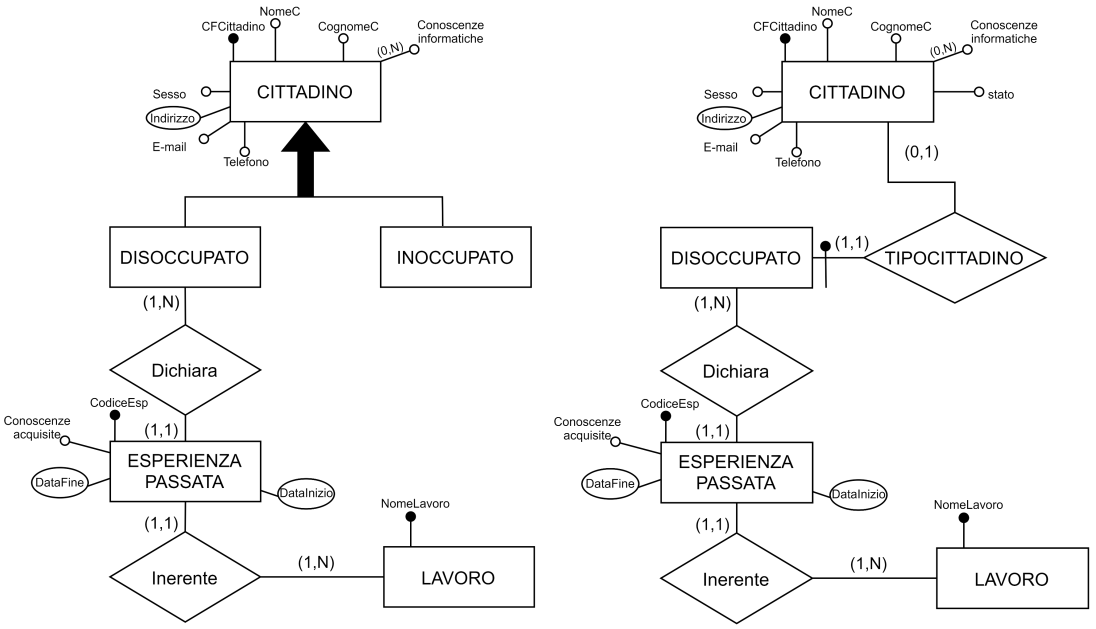


Figura 14 : Schema ristrutturato per la generalizzazione su CITTADINO

2.5 Accorpamento/partizionamento di relationship

Osservando lo schema E-R si evince che occorre eliminare alcuni attributi multivalore.

All'interno dell'entità Cittadino abbiamo l'attributo multivalore CONOSCENZEINFORMATICHE che dobbiamo eliminare poiché non sarebbe possibile rappresentarlo in fase di traduzione, quindi creiamo un'entità CONOSCENZEINFORMATICHE da collegare a cittadino mediante la relazione ASSERISCE.

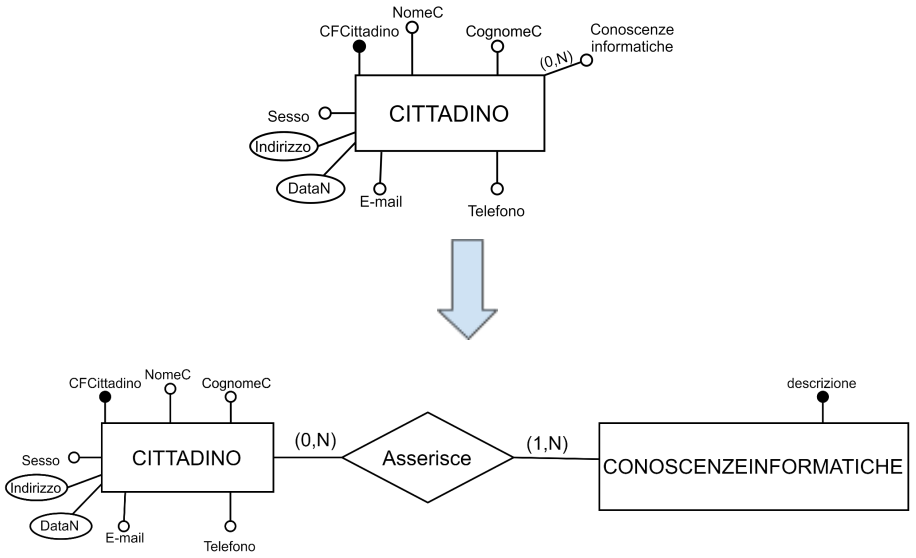


Figura 15 : Schema ristrutturato per l'attributo multivalore CONOSCENZEINFORMATICHE

All'interno dell'entità ESPERIENZAPASSATE abbiamo l'attributo multivalore CONOSCENZEACQUISITE che dobbiamo eliminare poiché non sarebbe possibile rappresentarlo in fase di traduzione, quindi creiamo un'entità CONOSCENZEACQUISITE da collegare a ESPERIENZEPASSATE mediante la relazione DERIVA.

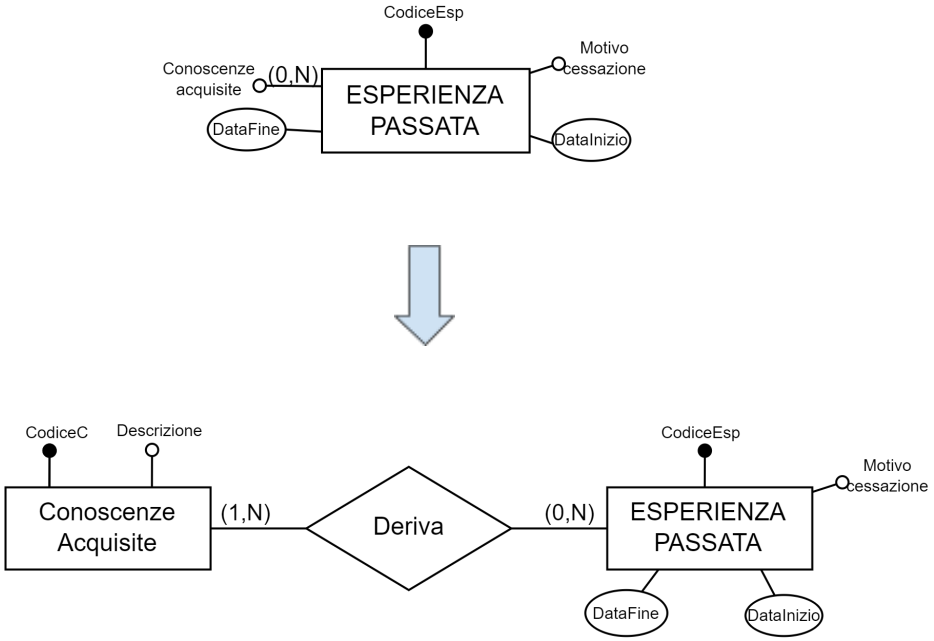


Figura 16 : Schema ristrutturato per l'attributo multivalore CONOSCENZEACQUISITE

2.6 Scelta degli identificatori primari

Osservando lo schema di figura 9 possiamo constatare che tutte le tabelle hanno un unico identificatore interno o esterno che diventa automaticamente l'identificatore principale, eccezion fatta per l'entità PERSONALE. Quest'ultima presenta due identificatori: il codice fiscale e la matricola. Tra i due identificatori è certamente preferibile scegliere il secondo. Infatti, dato che per rappresentare 150 impiegati mi bastano 3 cifre, la matricola comporta meno occupazione di memoria rispetto al codice fiscale che è composto da 16 caratteri alfanumerici.

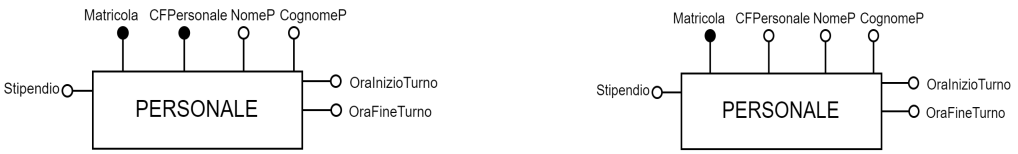


Figura 17 : Scelta degli identificatori per PERSONALE

Lo schema E – R completamente ristrutturato viene mostrato in seguito.

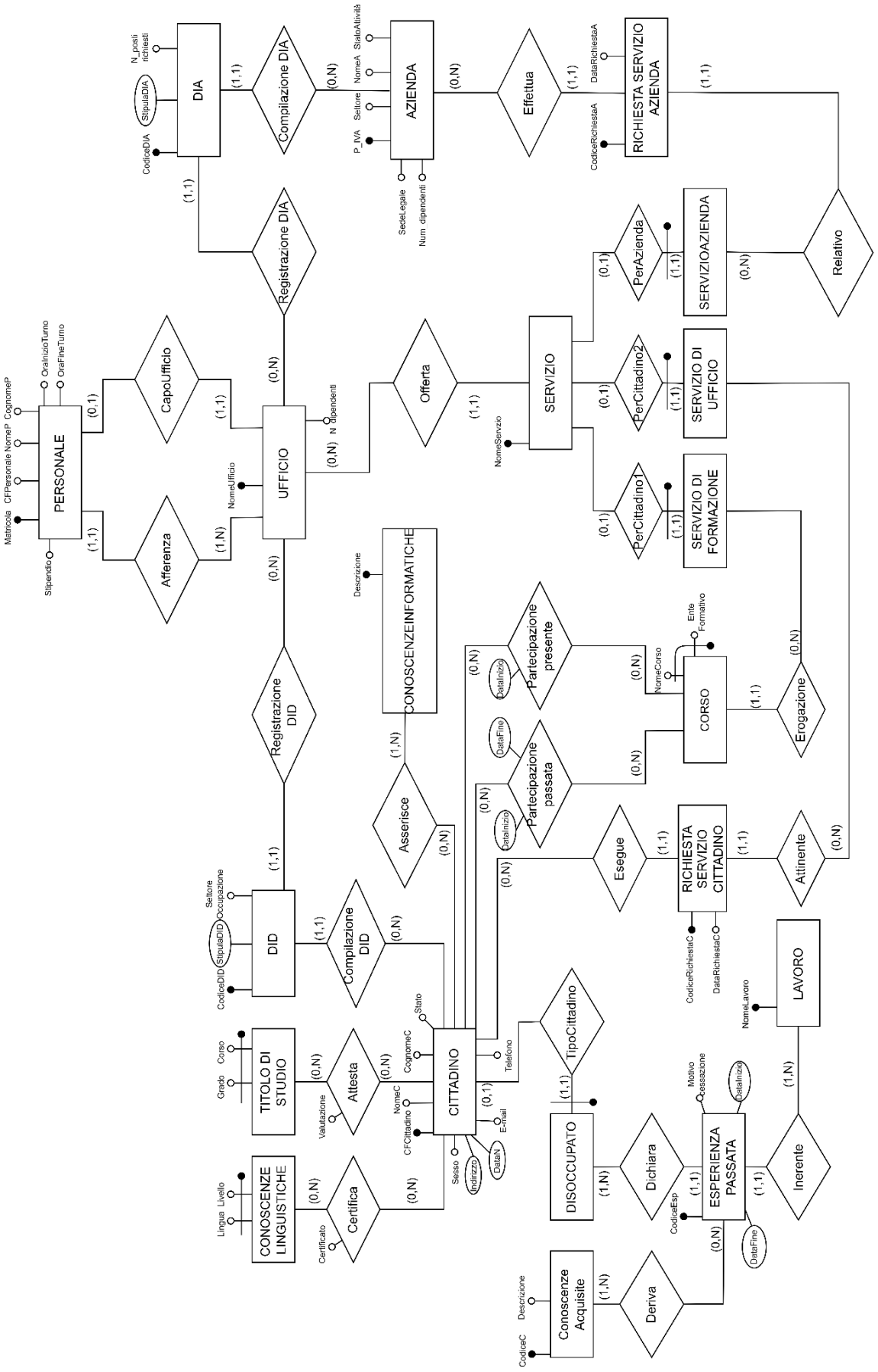


Figura 18 : Schema finale ristrutturato

3.Modello Relazionale

3.1 Traduzione di Entità

- **CITTADINO**(CFCittadino, NomeC, CognomeC, Sesso, DataN, NomeVia, Civico, Cap, Città, E-mail, Telefono, Stato)
- **CONOSCENZE_LINGUISTICHE**(Lingua, Livello)
- **TITOLO_DI_STUDIO**(Grado, Corso)
- **CONOSCENZE_INFORMATICHE**(Descrizione)
- **DISOCCUPATO**(CFCittadino)
  Vincolo di integrità referenziale fra l'attributo CFCittadino e la relazione Cittadino
- **ESPERIENZA_PASSATA**(CodiceEsp, Motivo_Cessazione, DataInizio, DataFine, NomeLavoro, Cittadino)
  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Disoccupato e tra l'attributo NomeLavoro e la relazione Lavoro
- **LAVORO**(NomeLavoro)
- **CONOSCENZE_ACQUISITE**(CodiceC, Descrizione)
- **DID**(CodiceDID, DataStipulaDID, Settore_Occupazione, Cittadino, Ufficio)
  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino e tra l'attributo Ufficio e la relazione Ufficio
- **UFFICIO**(NomeUfficio, CapoUfficio, N_Dipendenti)
  Vincolo di integrità referenziale fra l'attributo CapoUfficio e la relazione Personale
- **PERSONALE**(Matricola, Ufficio, CFPersonale, NomeP, CognomeP, Stipendio, OraInizioTurno, OraFineTurno)
  Vincolo di integrità referenziale fra l'attributo Ufficio e la relazione Ufficio
- **DIA**(CodiceDIA, DataStipulaDIA, N_PostiRichiesti, Ufficio, Azienda)

Vincolo di integrità referenziale fra l'attributo Ufficio e la relazione Ufficio e l'attributo Azienda e la relazione Azienda

- **AZIENDA**(P_IVA, NomeA, SedeLegale, StatoAttività, Num_dipendenti, Settore)
- **SERVIZIO**(NomeServizio, Ufficio)
  Vincolo di integrità referenziale fra l'attributo Ufficio e la relazione Ufficio
- **SERVIZIO_DI_FORMAZIONE**(NomeServizio)
  Vincolo di integrità referenziale fra l'attributo NomeServizio e la relazione Servizio
- **CORSO**(NomeCorso, EnteFormativo, TipoServizioErogato)
  Vincolo di integrità referenziale fra l'attributo TipoServizioErogato e la relazione Servizio_Di_Formazione
- **SERVIZIO_DI_UFFICIO**(NomeServizio)
  Vincolo di integrità referenziale fra l'attributo NomeServizio e la relazione Servizio
- **RICHIESTA_SERVIZIO_CITTADINO**(CodiceRichiestaC, DataRichiestaC, TipoServizioErogato, Cittadino)
  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino e tra l'attributo TipoServizioEorgato e la relazione Servizio_Di_Uffico
- **SERVIZIOAZIENDA**(NomeServizio)
  Vincolo di integrità referenziale fra l'attributo NomeServizio e la relazione Servizio
- **RICHIESTA_SERVIZIO_AZIENDA**(CodiceRichiestaA, DataRichiestaA, TipoServizioErogato, Azienda)
  Vincolo di integrità referenziale fra l'attributo Azienda e la relazione Azienda e tra l'attributo TipoServizioEorgato e la relazione ServizioAzienda

3.2 Traduzione di Relazioni

- **CERTIFICA**(Cittadino, Lingua, Livello, Certificato)

  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino e tra gli attributi Lingua,Livello e la relazione ConoscenzeLinguistiche.

- **ATTESTA**(Cittadino, Grado, Corso, Valutazione)

  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino e tra gli attributi Grado,Corso e la relazione TitoloDiStudio.

- **ASSERISCE**(Cittadino, Descrizione)

  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino e tra l'attributo Descrizione e la relazione ConoscenzeInformatiche.

- **PARTECIPAZIONE_PASSATA**(Cittadino, NomeCorso, EnteFormativo, DataInizio, DataFine)

  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino e tra gli attributi NomeCorso,EnteFormativo e la relazione Corso

- **PARTECIPAZIONE_PRESENTE**(Cittadino, NomeCorso, EnteFormativo, DataInizio)

  Vincolo di integrità referenziale fra l'attributo Cittadino e la relazione Cittadino, tra gli attributi NomeCorso e la relazione Corso

- **DERIVA**(CodiceC, CodiceEsp)

  Vincolo di integrità referenziale fra l'attributo CodiceC e la relazione Conoscenze_Acquisite e tra l'attributo CodiceEsp e la relazione Esperienza_Passata.

4.Implementazione delle operazioni

4.1 Creazione delle tabelle

CITTADINO:

```
CREATE TABLE public."Cittadino"(
      "CFCittadino" character varying(16)NOT NULL,
      "NomeC" character varying(20),
      "CognomeC" character varying(20),
      "Sesso" character varying(1),
      "DataN" date,
      "NomeVia" character varying(50),
      "Civico" numeric(4,0),
      "Cap" numeric(10,0),
      "Citta" character varying(20),
      "Email" character varying(20),
      "Telefono" numeric(15,0),
      CONSTRAINT "Cittadino_pkey" PRIMARY KEY ("CFCittadino"),
      CONSTRAINT "Cittadino_DataN_check"
      CHECK((datediff('hour'::character varying
      "DataN"::timestamp without time zone,
      CURRENT_DATE::timestamp without time zone) / 8766) >
      17),
      CONSTRAINT "Cittadino_DataN_check1" CHECK
      ((datediff('hour'::character varying, "DataN"::timestamp
      without time zone, CURRENT_DATE::timestamp without time
      zone) / 8766) < 66)
      CONSTRAINT cittadino_cf_check
      CHECK  (char_length(cfcittadino) = 16) NOT VALID,
      CONSTRAINT cittadino_cap_check
      CHECK (char_length(cap) = 5) NOT VALID

)
```

CONOSCENZE_LINGUISTICHE:

```
CREATE TABLE public."Conoscenze_Linguistiche"(
      "Lingua" character varying(20) NOT NULL unique,
      "Livello" character varying(3) NOT NULL unique,
      CONSTRAINT "Conoscenze_Linguistiche_pkey"
      PRIMARY KEY ("Lingua", "Livello")
)
```

TITOLO_DI_STUDIO:

```
CREATE TABLE public."Titolo_di_studio"(
      "Grado" character varying(20) NOT NULL unique,
      "Corso" character varying(20) NOT NULL unique,
      CONSTRAINT "Titolo_di_studio_pkey"
      PRIMARY KEY ("Grado", "Corso")
)
```

CONOSCENZE_INFORMATICHE:

```
CREATE TABLE  public."Conoscenze_informatiche"(
      "Descrizione" character varying(200) NOT NULL,
      CONSTRAINT "Conoscenze_informatiche_pkey"
      PRIMARY KEY ("Descrizione")
)
```

DISOCCUPATO:

```
CREATE TABLE public."Disoccupato"(
"CFCittadino" character varying(16) NOT NULL,
CONSTRAINT "Disoccupato_pkey"
PRIMARY KEY ("CFCittadino"),
CONSTRAINT "Disoccupato_CFCittadino_fkey"
FOREIGN KEY ("CFCittadino")
      REFERENCES public."Cittadino" ("CFCittadino")
      MATCH SIMPLE
      ON UPDATE NO ACTION
      ON DELETE CASCADE)
```

ESPERIENZA_PASSATA:

```
CREATE TABLE public."Esperienza_passata"(
      "CodiceEsp" numeric(20,0) NOT NULL,
      "Motivo_cessazione" character varying(50),
      "DataInizio" date,
      "DataFine" date,
      "NomeLavoro" character varying(20),
      "Cittadino" character varying(16),
      CONSTRAINT "Esperienza_passata_pkey"
      PRIMARY KEY ("CodiceEsp"),
      CONSTRAINT "Esperienza_passata_Cittadino_fkey"
      FOREIGN KEY ("Cittadino")
            REFERENCES public."Cittadino" ("CFCittadino")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT "Esperienza_passata_NomeLavoro_fkey"
      FOREIGN KEY ("NomeLavoro")
            REFERENCES public."Lavoro" ("NomeLavoro")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION
      CONSTRAINT "Esperienza_passata_check"
      CHECK ("DataFine" >  "DataInizio")
)
```

LAVORO:

```
CREATE TABLE  public."Lavoro"(
      "NomeLavoro" character varying(20) NOT NULL,
      CONSTRAINT "Lavoro_pkey" PRIMARY KEY ("NomeLavoro")
)
```

CONOSCENZE_ACQUISITE:

```
CREATE TABLE public."Conoscenze_acquisite"(
      "CodiceC" numeric(20,0) NOT NULL,
      "Descrizione" character varying(50),
      CONSTRAINT "Conoscenze_acquisite_pkey"
      PRIMARY KEY ("CodiceC")
)
```

DID:

```
CREATE TABLE public."Did"(
      "CodiceDID" numeric(20,0) NOT NULL,
      "DataStipulaDID" date,
      "Settore_occupazione" character varying(50),
      "Cittadino" character varying(16),
      "Ufficio" character varying(20),
      CONSTRAINT "Did_pkey" PRIMARY KEY ("CodiceDID"),
      CONSTRAINT "Did_Cittadino_fkey" FOREIGN KEY
      ("Cittadino")
            REFERENCES public."Cittadino" ("CFCittadino")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT "Did_Ufficio_fkey" FOREIGN KEY ("Ufficio")
            REFERENCES public."Ufficio" ("NomeUfficio")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION,
      CONSTRAINT "Did_DataStipulaDID_check"
      CHECK ("DataStipulaDID" <= CURRENT_DATE)
)
```

UFFICIO:

```
CREATE TABLE public.ufficio(
      nomeufficio character varying(50) NOT NULL,
      n_dipendenti numeric(100,0),
```

```
      capoufficio numeric(20,0) NOT NULL,
      CONSTRAINT ufficio_pkey PRIMARY KEY (nomeufficio),
      CONSTRAINT ufficio_capoufficio_fkey
      FOREIGN KEY (capoufficio)
            REFERENCES public.personale (matricola)
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION
)
```

PERSONALE:

```
CREATE TABLE public.personale(
      matricola numeric(3,0) NOT NULL,
      nomep character varying(20),
      cognomep character varying(20),
      stipendio numeric(10,0),
      orainizioturno time with time zone,
      orafineturno time with time zone,
      cfpersonale character varying(16),
      ufficio character varying(50),
      CONSTRAINT personale_pkey PRIMARY KEY (matricola),
      CONSTRAINT personale_ufficio_fkey FOREIGN KEY (ufficio)
            REFERENCES public.ufficio (nomeufficio) MATCH
            SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT personale_check
      CHECK (orafineturno > orainizioturno),
      CONSTRAINT personale_cf_check CHECK
      (char_length(cfpersonale::text) = 16)
)
```

DIA:

```
CREATE TABLE public.dia(
      codicedia numeric(20,0) NOT NULL,
```

```
      datastipuladia date,
      n_postirichiesti numeric(100,0),
      ufficio character varying(20),
      azienda numeric(11,0),
      CONSTRAINT dia_pkey PRIMARY KEY (codicedia),
      CONSTRAINT dia_azienda_fkey FOREIGN KEY (azienda)
            REFERENCES public.azienda (p_iva)
            MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE,
      CONSTRAINT dia_datastipuladia_check
      CHECK (datastipuladia <= CURRENT_DATE),
      CONSTRAINT dia_ufficio_check
      CHECK (ufficio::text ~~ 'Assunzione%'::text)
)
```

AZIENDA:

```
CREATE TABLE public.azienda(
      p_iva numeric NOT NULL,
      nomea character varying(20),
      sedelegale character varying(20),
      statoattivita character varying(20),
      num_dipendenti numeric(100,0),
      settore character varying(100),
      CONSTRAINT azienda_pkey PRIMARY KEY (p_iva)
)
```

SERVIZIO:

```
CREATE TABLE public.servizio(
      nomeservizio character varying(30) NOT NULL,
      ufficio character varying(20),
      CONSTRAINT servizio_pkey PRIMARY KEY (nomeservizio),
      CONSTRAINT ufficio_servizio_fkey FOREIGN KEY (ufficio)
            REFERENCES public.ufficio (nomeufficio)
            MATCH SIMPLE
            ON UPDATE CASCADE
```

```
            ON DELETE CASCADE
)
```

SERVIZIO_DI_FORMAZIONE:

```
CREATE TABLE public.servizio_di_formazione(
      nomeservizio character varying(50) NOT NULL,
      CONSTRAINT servizio_di_formazione_pkey
      PRIMARY KEY (nomeservizio),
      CONSTRAINT servizio_di_formazione_nomeservizio_fkey
      FOREIGN KEY (nomeservizio)
            REFERENCES public.servizio (nomeservizio)
            MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE
)
```

CORSO:

```
CREATE TABLE public.corso(
      nomecorso character varying(20) NOT NULL,
      enteformativo character varying(20) NOT NULL,
      tiposervizioerogato character varying(30),
      CONSTRAINT corso_pkey
      PRIMARY KEY (nomecorso, enteformativo),
      CONSTRAINT corso_enteformativo_key
      UNIQUE (enteformativo),
      CONSTRAINT corso_nomecorso_key UNIQUE (nomecorso),
      CONSTRAINT corso_tiposervizioerogato_fkey
      FOREIGN KEY (tiposervizioerogato)
      REFERENCES public.servizio_di_formazione(nomeservizio)
            MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE
)
```

SERVIZIO_DI_UFFICIO:

```
CREATE TABLE public.servizio_di_ufficio(
      nomeservizio character varying(30) NOT NULL,
      CONSTRAINT servizio_di_ufficio_pkey
      PRIMARY KEY (nomeservizio),
      CONSTRAINT servizio_di_ufficio_nomeservizio_fkey
      FOREIGN KEY (nomeservizio)
            REFERENCES public.servizio (nomeservizio)
            MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE
)
```

RICHIESTA_SERVIZIO_CITTADINO:

```
CREATE TABLE public.richiesta_servizio_cittadino(
      codicerichiestac numeric(20,0) NOT NULL,
      datarichiestac date,
      tiposervizioerogato character varying(30),
      cittadino character varying(16),
      CONSTRAINT "primary key" PRIMARY KEY (codicerichiestac),
      CONSTRAINT richiesta_servizio_cittadino_cittadino_fkey
      FOREIGN KEY (cittadino)
            REFERENCES public.cittadino (cfcittadino)
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT
      richiesta_servizio_cittadino_tiposervizioerogato_fkey
      FOREIGN KEY (tiposervizioerogato)
            REFERENCES public.servizio_di_ufficio
            (nomeservizio)
            MATCH SIMPLE
            ON UPDATE CASCADE
```

```
            ON DELETE CASCADE,
      CONSTRAINT
      richiesta_servizio_cittadino_datarichiestac_check
      CHECK (datarichiestac <= CURRENT_DATE))
```

SERVIZIO_AZIENDA:

```
CREATE TABLE public.servizio_azienda(
      nomeservizio character varying(50) NOT NULL,
      CONSTRAINT servizio_azienda_pkey
      PRIMARY KEY (nomeservizio),
      CONSTRAINT servizio_azienda_nomeservizio_fkey
      FOREIGN KEY (nomeservizio)
            REFERENCES public.servizio (nomeservizio)
            MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE
)
```

RICHIESTA_SERVIZIO_AZIENDA:

```
CREATE TABLE public.richiesta_sevizio_azienda(
      codicerichiestaa numeric(30,0) NOT NULL,
      datarichiestaa date,
      tiposervizioerogato character varying(50),
      azienda numeric(11,0),
      CONSTRAINT richiesta_sevizio_azienda_pkey
      PRIMARY KEY (codicerichiestaa),
      CONSTRAINT richiesta_sevizio_azienda_azienda_fkey
      FOREIGN KEY (azienda)
            REFERENCES public.azienda (p_iva) MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE,
      CONSTRAINT
      richiesta_sevizio_azienda_tiposervizioerogato_fkey
      FOREIGN KEY (tiposervizioerogato)
            REFERENCES public.servizio_azienda (nomeservizio)
            MATCH SIMPLE
            ON UPDATE CASCADE
```

```
            ON DELETE CASCADE,
      CONSTRAINT
richiesta_sevizio_azienda_datarichiestaa_check
      CHECK (datarichiestaa <= CURRENT_DATE)
)
```

CERTIFICA:

```
CREATE TABLE public.certifica(
      cittadino character varying(16) NOT NULL,
      lingua character varying(20) NOT NULL,
      livello character varying(2) NOT NULL,
      certificato boolean,
      CONSTRAINT certifica_pkey
      PRIMARY KEY (cittadino, lingua, livello),
      CONSTRAINT certifica_cittadino_fkey
      FOREIGN KEY (cittadino)
            REFERENCES public.cittadino (cfcittadino)
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT certifica_lingua_livello_fkey
      FOREIGN KEY (lingua, livello)
            REFERENCES public.conoscenze_linguistiche(lingua,
            livello)
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION
)
```

ATTESTA:

```
CREATE TABLE public."Attesta"(
      "Cittadino" character varying(16) NOT NULL,
      "Grado" character varying(20) NOT NULL,
      "Corso" character varying(20) NOT NULL,
      "Valutazione" numeric(3,0),
      CONSTRAINT "Attesta_pkey"
      PRIMARY KEY ("Grado", "Cittadino", "Corso"),
```

```
      CONSTRAINT "Attesta_Cittadino_fkey"
      FOREIGN KEY ("Cittadino")
            REFERENCES public."Cittadino" ("CFCittadino")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT "Attesta_Corso_fkey" FOREIGN KEY ("Corso")
            REFERENCES public."Titolo_di_studio" ("Corso")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION,
      CONSTRAINT "Attesta_Grado_fkey" FOREIGN KEY ("Grado")
            REFERENCES public."Titolo_di_studio" ("Grado")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION
)
```

ASSERISCE:

```
CREATE TABLE public."Asserisce"(
      "Cittadino" character varying(16) NOT NULL,
      "Descrizione" character varying(50) NOT NULL,
      CONSTRAINT "Asserisce_pkey"
      PRIMARY KEY ("Cittadino", "Descrizione"),
      CONSTRAINT "Asserisce_Cittadino_fkey"
      FOREIGN KEY ("Cittadino")
            REFERENCES public."Cittadino" ("CFCittadino")
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT "Asserisce_Descrizione_fkey"
      FOREIGN KEY ("Descrizione")
            REFERENCES public."Conoscenze_informatiche
            ("Descrizione") MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE NO ACTION
```

```
)
```

PARTECIPAZIONE_PASSATA:

```
CREATE TABLE public.partecipazione_passata(
      cittadino character varying NOT NULL,
      nomecorso character varying NOT NULL,
      enteformativo character varying NOT NULL,
      datainizio date,
      datafine date,
      CONSTRAINT partecipazione_passata_pkey
      PRIMARY KEY (cittadino, nomecorso, enteformativo),
      CONSTRAINT partecipazione_passata_cittadino_fkey
      FOREIGN KEY (cittadino)
            REFERENCES public.cittadino (cfcittadino)
            MATCH SIMPLE
            ON UPDATE NO ACTION
            ON DELETE CASCADE,
      CONSTRAINT
      partecipazione_passata_nomecorso_enteformativo_fkey
      FOREIGN KEY (enteformativo, nomecorso)
            REFERENCES public.corso (enteformativo, nomecorso)
            MATCH SIMPLE
            ON UPDATE CASCADE
            ON DELETE CASCADE,
      CONSTRAINT partecipazione_passata_check
      CHECK (datafine > datainizio)
)
```

DERIVA:

```
CREATE TABLE public."Deriva"(
      "CodiceC" numeric(20,0) NOT NULL,
```

```
    "CodiceEsp" numeric(20,0) NOT NULL,
    CONSTRAINT "Deriva_pkey"
    PRIMARY KEY ("CodiceC", "CodiceEsp"),
    CONSTRAINT "Deriva_CodiceC_fkey" FOREIGN KEY ("CodiceC")
        REFERENCES
        public."Conoscenze_acquisite"("CodiceC")
        MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE NO ACTION,
    CONSTRAINT "Deriva_CodiceEsp_fkey"
    FOREIGN KEY ("CodiceEsp")
        REFERENCES public."Esperienza_passata("CodiceEsp")
        MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE
)
```

PARTECIPAZIONE_PRESENTE:

```
CREATE TABLE public.partecipazione_presente(
    cittadino character varying(16) NOT NULL,
    nomecorso character varying(50) NOT NULL,
    datainizio date,
    enteformativo character varying(20),
    CONSTRAINT partecipazione_presente_pkey
    PRIMARY KEY (cittadino, nomecorso),
    CONSTRAINT partecipazione_presente_cittadino_fkey
    FOREIGN KEY (cittadino)
        REFERENCES public.cittadino (cfcittadino)
        MATCH SIMPLE
        ON UPDATE NO ACTION
        ON DELETE CASCADE,
    CONSTRAINT
    partecipazione_presente_nomecorso_enteformativo_fkey
    FOREIGN KEY (enteformativo, nomecorso)
        REFERENCES public.corso (enteformativo, nomecorso)
        MATCH SIMPLE
```

```
            ON UPDATE CASCADE
            ON DELETE CASCADE
)
```

4.2 Creazione delle query

OP1: Aggiunta di un nuovo cittadino indicando tutti i suoi dati

```
INSERT INTO CITTADINO ("cfcittadino", "nomec", "cognomec",
"sesso", "datan", "nomevia", "civico" , "citta" , "email",
"telefono", "cap", "stato_inoccupato")
VALUES ('DCRFNC80D45H456T', 'De Caro', 'Francesco', 'm',
'1980-04-05','Via Betarice', 23, 'Perugia',
'fra_dec@libero.it', 3425723765 , '06250', true );
```

OP2: Aggiunta di una nuova azienda indicando i suoi dati

```
INSERT INTO AZIENDA ("p_iva", "nomea", sedelegale,
"statoattivita", "num_dipendenti", "settore")
VALUES(02633000423, 'Fusiera S.p.a', 'Ancona', 'Attivo', 58,
'Fonditoria');
```

OP3: Aggiunta di un nuovo ufficio

```
CREATE OR REPLACE FUNCTION public.aggufficio(
nomeuff character varying,
capouff numeric)
    RETURNS void
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
declare conta int;
declare nomeu character varying;
BEGIN
if(capouff is null) then
INSERT INTO ufficio VALUES ( nomeuff, 0, capouff);
elsif NOT EXISTS(SELECT ufficio.capoufficio FROM ufficio
WHERE ufficio.capoufficio= capouff) THEN
```

```
select nomeufficio into nomeu from personale,ufficio where
matricola=capouff and personale.ufficio=ufficio.nomeufficio;
select n_dipendenti into conta from personale,ufficio where
matricola=capouff and personale.ufficio=ufficio.nomeufficio;
update ufficio set n_dipendenti=conta-1 where
nomeufficio=nomeu;
INSERT INTO ufficio VALUES ( nomeuff, 0, capouff);
update personale set ufficio=nomeuff where matricola=capouff;
update ufficio set n_dipendenti=1 where
ufficio.nomeufficio=nomeuff;
ELSE
raise exception 'il capo ufficio inserito già afferisce ad un
altro ufficio';
END IF;
END
$BODY$;
```

OP4: Aggiunta di un nuovo corso

```
INSERT INTO CORSO ("nomecorso", "enteformativo",
"tiposervizioerogato")
VALUES ('Beverage specialist', 'Business school', 'Corso di
formazione');
```

OP5: Aggiunta di un nuovo dipendente indicato i suoi dati

```
CREATE FUNCTION aggPersonale (mat numeric, np character
varying, cp character varying, stip numeric, orait time with
time zone, oraft time with time zone, cfp character varying,
uff character varying)
RETURNS void AS
$BODY$
declare nome int;
BEGIN
SELECT n_dipendenti into nome from ufficio where
uff=ufficio.nomeufficio;
```

```
INSERT INTO personale VALUES (mat, np, cp, stip, orait,
oraft,
cfp, uff);
UPDATE ufficio SET n_dipendenti=nome+1 where nomeufficio=uff;
END
$BODY$
LANGUAGE plpgsql VOLATILE;
```

OP6: Permettere al cittadino la richiesta di un'occupazione mediante DID

```
INSERT INTO DID ("codicedid", "datastipuladid",
"settore_occupazione", "cittadino", "ufficio")
VALUES (000001, '2017-09-11', 'Muratura', 'CVRRNS91D23G146W',
'Assunzione');
```

OP7: Permettere alle aziende la ricerca di personale mediante DIA

```
INSERT INTO DIA("codicedia", "datastipuladia",
"n_postirichiesti", "ufficio", "azienda")
VALUES (7,'2021-11-01',5,'Assunzione',78945612302);
```

OP8: Aggiungi esperienza lavorativa al cittadino

```
INSERT INTO esperienza_passata values(7, 'Licenziamento',
'2019-12-01', '2020-09-11', 'Fabbro','BRECLL87R19D765E')
```

OP9: Modifica dati relativi al numero dei dipendenti di un'azienda

```
UPDATE azienda set num_dipendenti = 12
WHERE p_iva = 12323568974;
```

OP10: Modifica capo di un ufficio

```
UPDATE ufficio set capoufficio=123
WHERE nomeufficio = 'Accoglienza' ;
```

OP11: Modifica dati relativi al personale

```
UPDATE personale set stipendio = 1200
WHERE matricola = 123;
```

OP12: Modifica stato di attività di un'azienda

```
UPDATE azienda set statoattivita = 'non attivo'
WHERE p_iva = 12633000423;
```

OP13: Elimina un cittadino

```
DELETE FROM cittadino
WHERE cfcittadino = 'ALBTMS78H15P536T';
```

OP14: Elimina un corso

```
DELETE FROM corso
WHERE NomeCorso = 'Beverage specialist';
```

OP15: Elimina un dipendente

```
CREATE OR REPLACE FUNCTION public.eliminapersonale(
mat numeric)
    RETURNS void
    LANGUAGE 'plpgsql'
    COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
declare nome int;
declare uff varchar;
BEGIN
select ufficio into uff from personale where
personale.matricola=mat;
select n_dipendenti into nome from ufficio where
ufficio.nomeufficio=uff;
update ufficio set n_dipendenti=nome-1 where nomeufficio=uff;
delete from personale where personale.matricola=mat;
END
```

```
$BODY$;
```

OP16: Stampa tutti i servizi per i cittadini

```
SELECT servizio_di_formazione.nomeservizio
FROM servizio_di_formazione
UNION
SELECT servizio_di_ufficio.nomeservizio
FROM servizio_di_ufficio
```

OP17: Stampa tutti i servizi per le aziende

```
SELECT servizio_azienda.nomeservizio
FROM servizio_azienda
```

OP18: Per ogni ufficio visualizzare il capo e il numero di impiegati

```
SELECT nomeufficio, n_dipendenti, cognomep as capoUfficio
FROM ufficio, personale
WHERE capoufficio=matricola
```

OP19: Visualizzare la data di fine rapporto lavorativo dei cittadini disoccupati

```
SELECT nomec AS nome, cognomec AS cognome, datafine
FROM esperienza_passata, cittadino
WHERE cittadino=cfcittadino AND datafine IN (SELECT
                                        MAX(datafine)
                                        FROM esperienza_passata
                                        GROUP BY cittadino)
```

OP20: Contare il numero di posti lavorativi disponibili in un certo settore

```
SELECT azienda.settore, SUM(n_postirichiesti) as
numero_posti_richiesti
FROM dia, azienda
WHERE dia.azienda = azienda.p_iva AND
azienda.settore = 'Elettronica'
```

```
GROUP BY azienda.settore
```

OP21: Contare i posti lavorativi disponibili per ogni azienda

```
SELECT azienda.nomea, SUM(n_postirichiesti) AS
numero_posti_richiesti
FROM dia, azienda
WHERE dia.azienda = azienda.p_iva
GROUP BY azienda.nomea
```

OP22: Selezionare i titoli di studio dei cittadini in cerca di occupazione per un dato settore

```
SELECT cittadino.nomec, cittadino.cognomec, attesta.grado,
attesta.corso
FROM cittadino, attesta, did
WHERE did.cittadino = attesta.cittadino AND did.cittadino =
cittadino.cfcittadino AND did.settore_occupazione='Edilizia'
```

OP23: Mostrare codice fiscale, nome e cognome dei cittadini che hanno avuto più esperienze lavorative rispetto agli altri

```
SELECT cfcittadino, nomec, cognomec
FROM cittadino
WHERE cfcittadino in(
SELECT cittadino
FROM esperienza_passata GROUP BY cittadino
HAVING COUNT(*)>= ALL (SELECT COUNT (*)
                       FROM esperienza_passata
                            GROUP BY cittadino))
```

OP24: Mostrare il codice fiscale dei cittadini che hanno richiesto più di 1 servizio nel 2021

```
create or replace view v1 as
SELECT codicerichiestac, cittadino
FROM richiesta_servizio_cittadino as RSC
WHERE RSC.datarichiestac between '2021-01-01' and
'2021-12-31';
```

```
SELECT cittadino, nomec, cognomec,
COUNT (codicerichiestac) as ServiziRichiesti
FROM v1 join cittadino as citt
on v1.cittadino = citt.cfcittadino
GROUP BY cittadino, nomec, cognomec
HAVING COUNT (codicerichiestac) > 1
```

OP25: Mostrare l'ufficio che spende di meno in stipendi per il proprio personale ad eccezione del capo

```
create or replace view v2 as
SELECT matricola,ufficio, stipendio
FROM personale
WHERE matricola <> ALL (SELECT capoufficio
                  FROM ufficio)

create or replace view stipendio_erogato_ufficio (uff, spesa)
as
SELECT ufficio, sum (stipendio)
FROM v2
GROUP BY ufficio


SELECT uff, spesa
FROM stipendio_erogato_ufficio
WHERE spesa = (SELECT min (spesa)
          FROM stipendio_erogato_ufficio)
```

OP26: Contare i cittadini in cerca di occupazione per settore

```
SELECT did.settore_occupazione, count(did.cittadino) as
numero_cittadini
FROM did
GROUP BY did.settore_occupazione
```

OP27: Mostrare i corsi di formazione

```
SELECT nomecorso, enteformativo as Ente
FROM corso
```

OP28: Visualizzare nome e cognome dei cittadini che hanno frequentato dei corsi

```
select nomec, cognomec, nomecorso
from cittadino, partecipazione_passata
where cfcittadino = cittadino
```

OP29: Visualizzare gli orari di lavoro del personale

```
SELECT nomep, cognomep, orainizioturno, orafineturno
FROM personale
```

4.3 Funzioni utilizzate

Funzione che calcola la differenza tra una data passata e la data corrente

```
CREATE OR REPLACE FUNCTION datediff (date_part VARCHAR(30),
start_t TIMESTAMP, end_t TIMESTAMP)
  RETURNS INT AS $diff$
  DECLARE
years INT = 0;
days INT = 0;
hours INT = 0;
minutes INT = 0;
  BEGIN
years = DATE_PART('year', end_t) - DATE_PART('year',
start_t);
IF date_part IN ('y', 'yr', 'yrs', 'year', 'years')  THEN
     RETURN years;
END IF;
IF date_part IN ('quarter', 'quarters', 'qtr', 'qtrs')  THEN
     RETURN years * 4 + (DATE_PART('quarter', end_t) -
DATE_PART('quarter', start_t));
END IF;
IF date_part IN ('month', 'months', 'mon', 'mons')  THEN
     RETURN years * 12 + (DATE_PART('month', end_t) -
DATE_PART('month', start_t));
END IF;
IF date_part IN ('week', 'weeks', 'w') THEN
     RETURN DATE_PART('day', (DATE_TRUNC('week', end_t) -
DATE_TRUNC('week', start_t)) / 7);
END IF;
days = DATE_PART('day', DATE_TRUNC('day', end_t) -
DATE_TRUNC('day', start_t));
IF date_part IN ('day', 'days', 'd') THEN
     RETURN days;
END IF;
hours = days * 24 + (DATE_PART('hour', end_t) -
DATE_PART('hour', start_t));
IF date_part IN ('hour', 'hours', 'h', 'hr', 'hrs') THEN
     RETURN hours;
END IF;
     minutes = hours * 60 + (DATE_PART('minute', end_t) -
DATE_PART('minute', start_t));
IF date_part IN ('minute', 'minutes', 'm', 'min', 'mins')
THEN
     RETURN minutes;
END IF;
```

```
IF date_part IN ('second', 'seconds', 's', 'sec', 'secs')
THEN
     RETURN minutes * 60 + (DATE_PART('second', end_t) -
DATE_PART('second', start_t));
END IF;
RETURN 0;
END;
$diff$ LANGUAGE plpgsql;
```

Funzione utilizzata per controllare se un cittadino è inoccupato o no

```
CREATE OR REPLACE FUNCTION public.check_disoccupato(
cf_cittadino character)
RETURNS boolean
     LANGUAGE 'plpgsql'
     COST 100
     VOLATILE PARALLEL UNSAFE
AS $BODY$
BEGIN
RETURN  (SELECT cittadino.stato_inoccupato FROM cittadino
WHERE cittadino.cfcittadino = cf_cittadino);
END
$BODY$;
```

Funzione utilizzata per eliminare il personale

```
CREATE OR REPLACE FUNCTION public.aggpersonale(mat numeric,
np character varying,cp character varying, stip numeric,
orait time with time zone,oraft time with time zone, cfp
character varying,
uff character varying)

RETURNS void
     LANGUAGE 'plpgsql'
     COST 100
     VOLATILE PARALLEL UNSAFE
AS $BODY$
declare
nome int;
BEGIN
SELECT n_dipendenti into nome from ufficio
WHERE uff=ufficio.nomeufficio;
INSERT INTO personale VALUES (mat, np, cp, stip, orait,
oraft,cfp, uff);
UPDATE ufficio SET n_dipendenti=nome+1 where nomeufficio=uff;
END
```

```
$BODY$;
```

Funzione utilizzata per aggiungere un nuovo ufficio

```
CREATE OR REPLACE FUNCTION public.aggufficio(nomeuff
character varying,capouff numeric)
RETURNS void
     LANGUAGE 'plpgsql'
     COST 100
     VOLATILE PARALLEL UNSAFE
AS $BODY$
declare conta int;
declare nomeu character varying;
BEGIN
IF(capouff is null) THEN
INSERT INTO ufficio VALUES ( nomeuff, 0, capouff);
ELSEIF NOT EXISTS(SELECT ufficio.capoufficio FROM ufficio
WHERE ufficio.capoufficio= capouff) THEN
SELCT nomeufficio INTO nomeu FROM personale,ufficio WHERE
matricola=capouff AND personale.ufficio=ufficio.nomeufficio;
SELECT n_dipendenti INTO conta FROM personale,ufficio WHERE
matricola=capouff and personale.ufficio=ufficio.nomeufficio;
UPDATE ufficio set n_dipendenti=conta-1 WHERE
nomeufficio=nomeu;
INSERT INTO ufficio VALUES ( nomeuff, 0, capouff);
UPDATE personale SET ufficio=nomeuff WHERE matricola=capouff;
UPDATE ufficio SET n_dipendenti=1 WHERE
ufficio.nomeufficio=nomeuff;
ELSE
raise exception 'il capo ufficio inserito già afferisce ad un
altro ufficio';
END IF;

END
$BODY$;
```

Funzione utilizzata per aggiungere il personale

```
CREATE OR REPLACE FUNCTION public.aggpersonale(
mat numeric,
np character varying,
cp character varying,
stip numeric,
orait time with time zone,
oraft time with time zone,
```

```
cfp character varying,
uff character varying)
     RETURNS void
     LANGUAGE 'plpgsql'
     COST 100
    VOLATILE PARALLEL UNSAFE
AS $BODY$
declare nome int;
BEGIN
SELECT n_dipendenti INTO nome FROM ufficio WHERE
uff=ufficio.nomeufficio;
INSERT INTO personale VALUES (mat, np, cp, stip, orait,
oraft, cfp, uff);
UPDATE ufficio SET n_dipendenti=nome+1 where nomeufficio=uff;
END
$BODY$;
```